Recurrent Neural Networks

The complete lecture: sequences and the hidden state, where the recurrence comes from, backpropagation through time in three views, why long sequences are hard, gated cells – all of it in TensorFlow/Keras and PyTorch

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for this lecture

Chapter 11 in one sitting, with every code example in TensorFlow/Keras or PyTorch:

1. **From feed-forward to recurrent networks**: what we give up, what a sequence model is asked to do, the hidden state.
2. **The architecture**: the simple RNN, folded and unrolled; the RNN as a dynamical system.
3. **Where the recurrence comes from**: a damped oscillator, Euler's method as a recurrence, the state-space form, and why the integrator matters.
4. **The layers in Keras and PyTorch**: what `SimpleRNN` and `nn.RNN` compute, line by line.
5. **Backpropagation through time, three ways**: the recursion of the chapter, the pre-activation error and the outer products, the path sum; a worked example checked with autograd; the discrete adjoint; what `loss.backward()` does.
6. **Why long sequences are hard**: the Jacobian product measured; eigenmodes, saturation and transient growth; truncated BPTT in practice; clipping; a network that learns the rule of an oscillator, and what a free rollout reveals.
7. **Long short-term memory and the GRU**: gates, the cell state, by how much it helps.
8. **Four applications**: a sine wave, MNIST row by row, spoken digits, and the adding problem.

Reading: Chapter 11 (all of it); Goodfellow et al. Chapter 10; Raschka et al. Chapter 15; Pascanu, Mikolov and Bengio (ICML 2013). The code is in `week45rnn.ipynb`.

What we have assumed so far, 1: independent samples

Almost everything since week 35 – the bias-variance decomposition, the bootstrap, cross-validation, the statistical toolbox of Chapter 2 – leans on the training examples being *independent and identically distributed*.

For a sequence this is simply false. The value of a detector trace at one sample is strongly informative about the next; that is the entire content of the data.

The consequence

A model that treats the samples as independent has thrown the signal away before it starts.

What we have assumed so far, 2: a fixed input size

A feed-forward network has a fixed input dimension. A convolutional network (week 44) has one too, up to the pooling that precedes its dense layers.

Sequences come in different lengths – a measurement that ran for ten seconds and one that ran for an hour, a spoken word of 0.3 s and one of 0.6 s – and we would like *one* model to handle all of them.

The move of this chapter

Week 44

We restricted the dense layer so that it respects the *spatial* structure of an image, and obtained *convolution*.

Today

We make the same kind of restriction for data with a *temporal* structure, and obtain *recurrence*: process one element at a time and carry a state forward, so that the same parameters serve a sequence of any length.

Sequences in the physical sciences

Ordered, correlated data are the norm rather than the exception:

- detector read-out streams and waveform traces;
- climate and seismic records;
- molecular-dynamics trajectories and spectroscopy scans;
- state estimation in control and feedback;
- the solutions of differential equations, which are time series by construction;
- sound – which we will classify today.

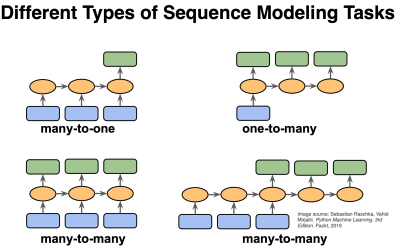
Sequential need not mean temporal (text, genomes), but our examples are time series. We write $x_{1:T} = (\mathrm{x}_1, \ldots, \mathrm{x}_T)$.

Three tasks, Eq. (11.1)

$$p(x_{1:T}), \qquad p(y_{1:T} \mid x_{1:T}), \qquad p(y \mid x_{1:T}).$$

- modelling the sequence itself;
- mapping a sequence to a sequence: filtering, forecasting (*many-to-many*);
- mapping a sequence to one label: classification, regime detection (*many-to-one*).

Today: the oscillator and the sine wave are many-to-many; MNIST, the spoken digits and the adding problem are many-to-one.



Figures in this part from Raschka and Mirjalili, *Python Machine Learning*.

What is a recurrent neural network?

In a feed-forward network the information flows in one direction only, from the input to the output.

A recurrent network also has connections pointing *backward*: the hidden layer is connected to *itself*. The hidden state at step t receives two inputs – the data $x^{(t)}$ and its own value $h^{(t-1)}$ from the previous step.

What this buys

A *memory* across time steps, and the same layer for the first element of a sequence and the thousandth.

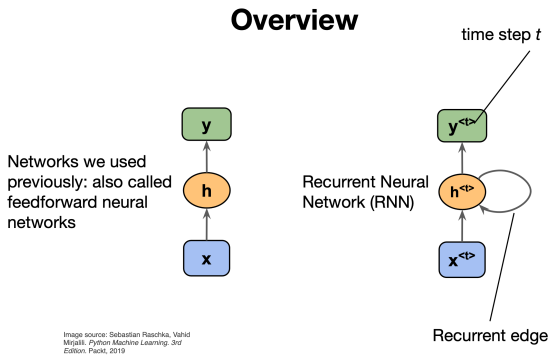


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

Fold and unfold: two drawings of the same computation

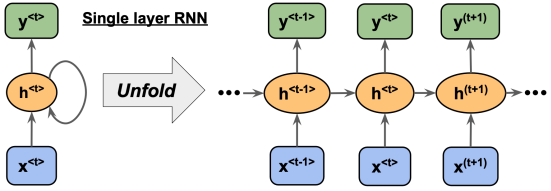


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition*. Packt, 2019

Folded (left)

A single layer with an arrow leaving it and returning to itself. This is how the network is *stored*: one copy of the weights.

Unrolled (right)

The same layer drawn once per time step. This is how the network is *executed* – and, as we will see, how it is *differentiated*.

The hidden state: a latent state-space model

The chain rule of probability factorises the first task exactly, Eq. (11.2),

$$p(x_{1:T}) = \prod_{t=1}^{T} p(\mathrm{x}_t \mid x_{1:t-1}),$$

exact and useless as it stands: the conditioning set grows without bound. What makes it tractable is the assumption that everything relevant in $x_{1:t-1}$ can be summarised in a finite-dimensional *state* h_t, updated recursively, Eq. (11.3):

$$\boxed{\mathrm{h}_t = f_\theta(\mathrm{h}_{t-1}, \mathrm{x}_t), \qquad \hat{\mathrm{y}}_t = g_\theta(\mathrm{h}_t).}$$

This should look familiar

Equation (11.3) is the structure of a *Markovian dynamical system*. The Kalman filter is the case where f_θ is linear and the noise Gaussian. A recurrent neural network is the case where f_θ and g_θ are neural networks – the update map is *learned*.

The simple RNN, Eq. (11.4)

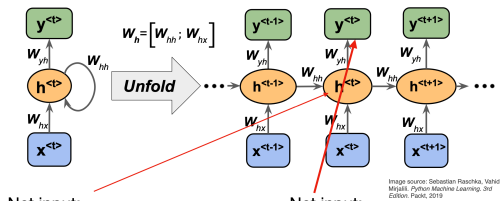
One affine map and one activation per step:

$$\mathrm{a}^{(t)} = \mathrm{U}\mathrm{x}^{(t)} + \mathrm{W}\mathrm{h}^{(t-1)} + \mathrm{b},$$
$$\mathrm{h}^{(t)} = \sigma_h\left(\mathrm{a}^{(t)}\right),$$
$$\mathrm{o}^{(t)} = \mathrm{V}\mathrm{h}^{(t)} + \mathrm{c},$$
$$\hat{\mathrm{y}}^{(t)} = \sigma_y\left(\mathrm{o}^{(t)}\right),$$

$\mathrm{U} \in \mathbb{R}^{n_h \times n_x}$ maps the input into the state, $\mathrm{W} \in \mathbb{R}^{n_h \times n_h}$ the state to itself, $\mathrm{V} \in \mathbb{R}^{n_y \times n_h}$ reads the state out; $\mathrm{h}^{(0)} = 0$.

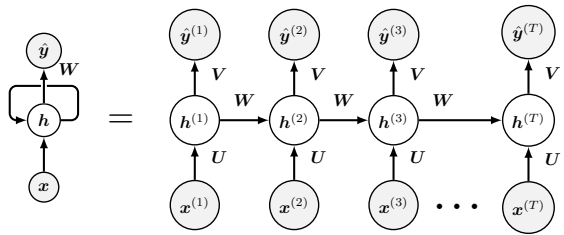
σ_h is almost always tanh (Part 5 says why); σ_y is the identity for regression, the softmax for classification.

Weight matrices in a single-hidden layer RNN



Raschka's notation: $\mathrm{U} = \mathrm{W}_{hx}$, $\mathrm{W} = \mathrm{W}_{hh}$, $\mathrm{V} = \mathrm{W}_{yh}$.

The same arrays at every step (Figure 11.1)



The parameters do not depend on t

Parameter sharing along the time axis, exactly as convolution shares parameters along the spatial axes (week 44): the parameter count is independent of the sequence length, and the model is equivariant to a shift in time.

Two more properties of Eq. (11.4)

The state is a bottleneck

Everything the network will ever know about $x_{1:t}$ when it predicts $\hat{y}^{(t+1)}$ must fit in the n_h numbers of $h^{(t)}$.

The cost accumulates over the sequence, Eq. (11.5)

$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} L^{(t)}, \qquad L^{(t)} = \tfrac{1}{2}\|\hat{y}^{(t)} - y^{(t)}\|^2$$

for regression, the cross entropy for classification. In the sequence-to-label case only $L^{(T)}$ is non-zero – the hardest case of all, as Part 5 will show.

Pause and think 1: count the parameters

Question

How many parameters has the simple RNN of Eq. (11.4) with $n_x = 1$, $n_h = 24$, $n_y = 1$ – the network we train on the oscillator in Part 5? How many would a dense network need if a sequence of length $T = 40$ were flattened into one input vector, with the same 24 hidden nodes? And what happens to each count when T becomes 400?

Pause and think 1: count the parameters

Question

How many parameters has the simple RNN of Eq. (11.4) with $n_x = 1$, $n_h = 24$, $n_y = 1$ – the network we train on the oscillator in Part 5? How many would a dense network need if a sequence of length $T = 40$ were flattened into one input vector, with the same 24 hidden nodes? And what happens to each count when T becomes 400?

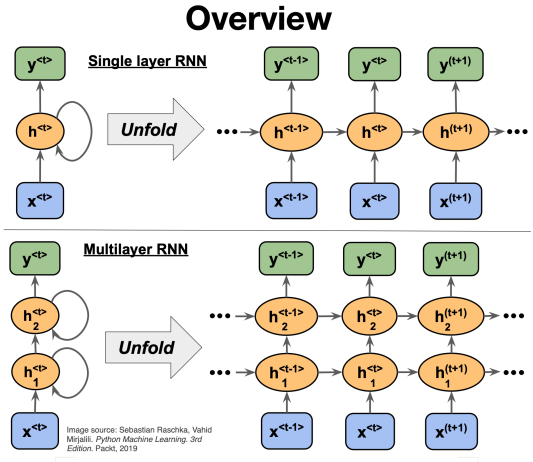
Answer

U: 24×1, W: $24 \times 24 = 576$, b: 24, V: 1×24, c: 1 – in total $n_h(n_x + n_h + 1) + n_y(n_h + 1) = 24 \cdot 26 + 25 = 649$, and T appears nowhere. The flattened dense network has $40 \times 24 + 24 + 24 + 1 = 1009$ parameters at $T = 40$ and 9649 at $T = 400$ – and must be *retrained* for every new T. The recurrent network has 649 either way. (Keras will print exactly 649 for this model; PyTorch prints 673 – Part 3 explains the difference.)

Stacking: a multilayer RNN

The hidden state of one recurrent layer can be the input sequence of the next. Each layer has its own U, W, b and is unrolled along the same time axis, so the unrolled graph becomes a grid: depth in layers downwards, depth in time sideways.

In the libraries: `num_layers=2` in `nn.RNN`, or two `SimpleRNN` layers with `return_sequences=True` in Keras.



The RNN as a discrete-time dynamical system (Section 11.2.1)

Set the input to zero. Equation (11.4) becomes an autonomous map, Eq. (11.6),

$$\mathsf{h}^{(t)} = F\left(\mathsf{h}^{(t-1)}\right) := \sigma_h\left(\mathsf{W}\mathsf{h}^{(t-1)} + \mathsf{b}\right).$$

Fixed points satisfy $\mathsf{h}^\star = F(\mathsf{h}^\star)$; their stability is decided by the Jacobian, Eq. (11.7),

$$\mathsf{J} = \left.\frac{\partial F}{\partial \mathsf{h}}\right|_{\mathsf{h}^\star} = \operatorname{diag}(\sigma_h'(\mathsf{a}^\star))\,\mathsf{W},$$

attracting if the spectral radius $\rho(\mathsf{J}) < 1$, repelling if $\rho(\mathsf{J}) > 1$. In two dimensions the map can have limit cycles, in three or more it can be chaotic.

Why this matters, twice

What a recurrent network can represent

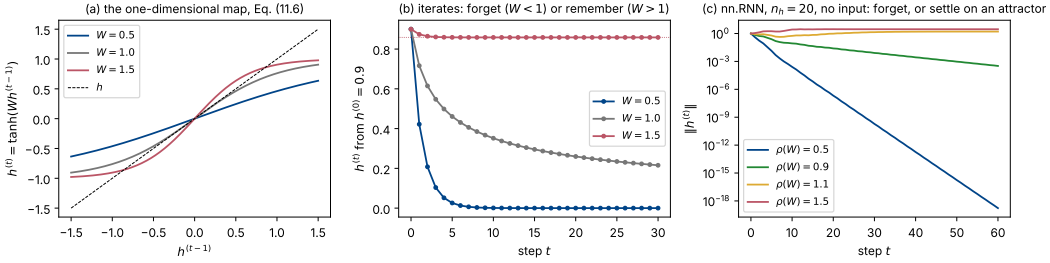
Oscillations, decay to a point attractor, chaotic wandering.

What it does with information

Strongly contracting dynamics forget the initial condition quickly and cannot carry information far; expanding dynamics are unstable.

Equation (11.7) is also exactly the quantity that will control the gradient in Part 5: the stability of the forward dynamics and the survival of the backward gradient are the *same question*.

The one-dimensional map, and an untrained `nn.RNN` (`week45rnn_dynamics.pdf`)



(a) $h \mapsto \tanh(Wh)$ against the diagonal: for $W < 1$ the only fixed point is 0, for $W > 1$ two new ones appear. (b) Iterates from $h^{(0)} = 0.9$: at $W = 0.5$ the state is gone after ten steps, at $W = 1.5$ it sits on $h^\star = \pm 0.8586$ for ever. (c) A PyTorch `nn.RNN` with $n_h = 20$, zero input, W scaled to a given spectral radius: at $\rho(\mathrm{W}) = 0.5$ the norm of the state falls by nineteen orders of magnitude in sixty steps; at $\rho > 1$ it settles on an attractor of order one.

Pause and think 2: forget or remember

Question

For the one-dimensional map $h^{(t)} = \tanh(Wh^{(t-1)})$ (no bias): show that $h^\star = 0$ is always a fixed point, decide when it is stable, and say what happens at $W = 1$. What does a network with $|W| < 1$ do with the information in $h^{(0)}$?

Pause and think 2: forget or remember

Question

For the one-dimensional map $h^{(t)} = \tanh(W h^{(t-1)})$ (no bias): show that $h^\star = 0$ is always a fixed point, decide when it is stable, and say what happens at $W = 1$. What does a network with $|W| < 1$ do with the information in $h^{(0)}$?

Answer

$\tanh(0) = 0$, so 0 is a fixed point for every W. Its Jacobian is $J = \tanh'(0)\, W = W$: stable for $|W| < 1$, unstable for $|W| > 1$. At $W = 1$ a *pitchfork bifurcation* produces two new stable fixed points $\pm h^\star$ with $h^\star = \tanh(W h^\star)$ – 0.66 at $W = 1.2$, 0.86 at $W = 1.5$, 0.96 at $W = 2$. With $|W| < 1$ every initial condition is driven to 0 geometrically, $|h^{(t)}| \le |W|^t |h^{(0)}|$: the network *forgets* its input at the rate $|W|^t$. In Part 5 this very number reappears as the size of the gradient that reaches step 0 from step t.

A differential equation (Section 11.3)

The recurrence of Eq. (11.4) can look arbitrary. It is not. Newton's equation for a damped harmonic oscillator, scaled to dimensionless time, Eq. (11.8):

$$\frac{\mathrm{d}^2x}{\mathrm{d}t^2} + \eta\frac{\mathrm{d}x}{\mathrm{d}t} + x(t) = F(t),$$

with η the damping and F an external force. As always we rewrite it as two coupled first-order equations, Eq. (11.9),

$$v = \frac{\mathrm{d}x}{\mathrm{d}t}, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = F(t) - \eta v(t) - x(t),$$

and integrate from (x_0, v_0) – by hand for $F = 0$, by Runge-Kutta in general.

Discretise, and the recurrence appears

Forward Euler for the velocity gives step $i+1$ from step i, Eq. (11.10):

$$v_{i+1} = v_i + \Delta t\,(F_i - \eta v_i - x_i) =: h_i(x_i, v_i, F_i)\,,$$

a function of the *current state* and the *current input*. But v_i was itself produced by the same rule one step earlier, Eq. (11.11), so substituting, Eq. (11.12), and renaming, Eq. (11.13),

$$v_{i+1} = h_i(x_i,\, h_{i-1},\, F_i) \qquad \Longrightarrow \qquad \mathrm{h}_i = h(\mathrm{x}_i, \mathrm{h}_{i-1})\,,$$

which is Eq. (11.3) exactly.

Position alone is not a state (Section 11.3.1)

Two oscillators through the same point

Two oscillators pass through the same position x at the same time, one moving left and the other right. Their next positions differ even if the force is identical. **Position alone is an incomplete description**; position and velocity together are a *state*: given (x_i, v_i) and the input F_i, the next (x_{i+1}, v_{i+1}) is determined.

What this says about the hidden state

The hidden state $\mathrm{h}^{(t)}$ of a recurrent network has to play this role for whatever process generated the data: it must carry *everything about the past that the future depends on*. If the data were produced by a second-order equation and the network is only given x, the network has to *reconstruct* the velocity inside $\mathrm{h}^{(t)}$ from consecutive inputs – which is what the oscillator experiment later in the lecture asks it to do.

The state-space form, Eqs. (11.14)–(11.17)

Euler for *both* components, with $\mathsf{s}_i = (x_i, v_i)^{\mathsf{T}}$:

$$x_{i+1} = x_i + \Delta t\, v_i, \qquad v_{i+1} = \underbrace{(1 - \eta\Delta t)}_{\text{recurrent weight}}\, v_i + \underbrace{\Delta t}_{\text{force weight}}\, F_i - \underbrace{\Delta t}_{\text{position coefficient}}\, x_i,$$

$$\mathsf{s}_{i+1} = \mathsf{A}\mathsf{s}_i + \mathsf{B}F_i, \qquad \mathsf{A} = \begin{pmatrix} 1 & \Delta t \\ -\Delta t & 1 - \eta\Delta t \end{pmatrix}, \quad \mathsf{B} = \begin{pmatrix} 0 \\ \Delta t \end{pmatrix}.$$

Unrolled, Eq. (11.16): $\mathsf{s}_n = \mathsf{A}^n \mathsf{s}_0 + \sum_{i=0}^{n-1} \mathsf{A}^{\,n-1-i} \mathsf{B} F_i$ – an input F_i reaches s_n through $n-1-i$ applications of A.

This *is* a linear recurrent network, Eq. (11.17)

Set σ_h = identity, $\mathsf{W} = \mathsf{A}$, $\mathsf{U} = \mathsf{B}$, $\mathsf{V} = (1, 0)$, $\mathsf{b} = \mathsf{c} = 0$: Eq. (11.4) with $\mathsf{h}^{(t)} = \mathsf{s}_t$ and $\mathsf{x}^{(t)} = F_t$ reproduces forward Euler exactly. The simple RNN is the same object with a non-linearity and *learned* coefficients. With $\Delta t = 0.05$, $\eta = 0.2$: $\mathsf{A} = \left(\begin{smallmatrix} 1 & 0.05 \\ -0.05 & 0.99 \end{smallmatrix}\right)$, eigenvalues $0.995 \pm 0.0498\, i$, modulus 0.9962.

The integrator itself can amplify errors (why RK4 makes the data)

Undamped, unforced: the exact motion is bounded

Forward Euler's matrix has eigenvalues $1 \pm i\Delta t$, modulus $\sqrt{1+\Delta t^2} = 1.00125$ at $\Delta t = 0.05$: the amplitude *grows* by a factor $e \approx 2.7$ over the 800 steps to $t = 40$.

Damped, $\eta = 0.2$

Euler decays by 0.9962 per step where the exact solution decays by $e^{-\eta \Delta t/2} = 0.9950$: by $t = 40$ the Euler amplitude is 2.7 times the true one.

Two lessons

- A network that learns the Euler update *perfectly* reproduces this artificial growth. The training data of this lecture are therefore generated by RK4, and a rollout is always compared with a trusted reference, so that an error in the learned dynamics can be told from an error already in the integrator.
- The modulus of the eigenvalues of the one-step map decides whether a perturbation grows or decays. Keep this in mind: it is exactly the quantity that will govern the *gradients* in Section 11.5.

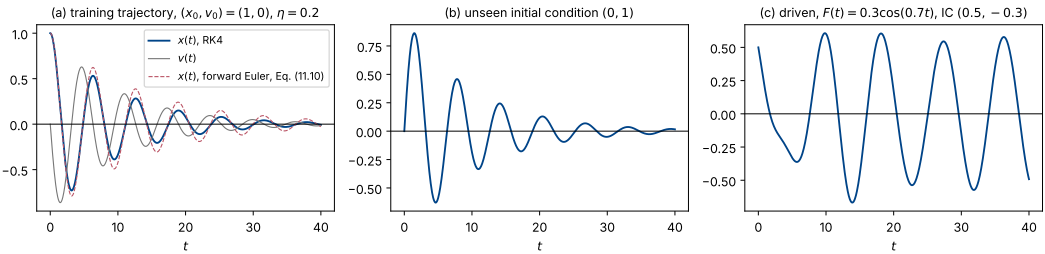
A recurrent network is a learned, non-linear time-stepper

The correspondence

- the hidden state plays the role of the integrator's internal state;
- W plays the role of $1 - \eta \Delta t$;
- the training data supply what the differential equation would otherwise have to tell us.

The output $\hat{y}_i$ is compared with a target through a cost – the target being a measurement, or the solution of the equation.

The data for today's experiment (`week45rnn_oscillator_data.pdf`)



Fourth-order Runge-Kutta with $\eta = 0.2$ on $t \in [0, 40]$, 800 points. Forward Euler at the same $\Delta t = 0.05$ (dashed) is visibly off by the second period – an error of 0.1 – a first hint of what a *learned* stepper can improve on. In Part 5 the network sees only the blue curve of (a); (b) and (c) are the tests.

Compare week 42: two ways to solve a differential equation with a network

Chapter 9 (week 42)

The equation was *known* and supplied the cost through its residual; no data were needed; the network learned a function of continuous time.

Chapter 11 (today)

The equation is *unknown* and the data supply the cost; the network learns a discrete update rule.

The first is for known physics and absent data, the second for present data and unknown physics.

Two libraries, one layer

Equation	TensorFlow/Keras	PyTorch
simple RNN, Eq. (11.4)	`layers.SimpleRNN(units)`	`nn.RNN(input_size, hidden_size)`
LSTM, Eqs. (11.49)–(11.52)	`layers.LSTM(units)`	`nn.LSTM(input_size, hidden_size)`
GRU, Eq. (11.56)	`layers.GRU(units)`	`nn.GRU(input_size, hidden_size)`
read-out $\mathbf{V}\mathbf{h} + \mathbf{c}$	`layers.Dense(n_y)`	`nn.Linear(hidden_size, n_y)`
clipping, Eq. (11.46)	`Adam(lr, clipnorm=1.0)`	`nn.utils.clip_grad_norm_(params, 1.0)`
BPTT	inside `model.fit`	`loss.backward()`

The one convention to remember: the data layout

Both libraries want a 3-tensor of shape `(samples, timesteps, features)` $= (N, T, n_x)$: N sequences, each T steps of n_x numbers. In PyTorch this needs `batch_first=True` (the default is (T, N, n_x)). A single time series is *cut into windows* to make the first axis.

Keras: a minimal recurrent model

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(None, 1)), # (T, n_x): T may vary, one feature per step
    layers.SimpleRNN(24, return_sequences=True), # Eq. (11.4): h_t for every t, n_h = 24
    layers.Dense(1), # V h_t + c, applied at every step
])
model.compile(optimizer=keras.optimizers.Adam(5e-3, clipnorm=1.0), loss="mse")
model.summary() # 24*(1+24+1) + 25 = 649 parameters
history = model.fit(X_train, Y_train, epochs=300, batch_size=4, verbose=0)
```

`X_train` has shape $(N, T, 1)$ and `Y_train` the shape $(N, T, 1)$ of the targets – here one target per step, the sequence-to-sequence case. Six lines; the next frame reads them one by one.

Keras: what each line does

`Input(shape=(None, 1))` the shape of *one* sample without the batch axis: T steps (`None` = any length) of $n_x = 1$ features.

`SimpleRNN(24)` one recurrent layer with $n_h = 24$: U (1×24, stored transposed as `kernel`), W (24×24, `recurrent_kernel`), b (`bias`); tanh by default; $\mathrm{h}^{(0)} = 0$.

`return_sequences=True` output all of $\mathrm{h}^{(1)}, \ldots, \mathrm{h}^{(T)}$, shape $(N, T, 24)$. With `False` (the default) only $\mathrm{h}^{(T)}$, shape $(N, 24)$ – the sequence-to-label case.

`Dense(1)` $\mathrm{V}\mathrm{h} + \mathrm{c}$. Applied to a 3-tensor it acts on the last axis, i.e. at every time step with the same V, c.

`compile(...)` the optimiser (Adam, week 38) and the cost; `clipnorm=1.0` is gradient clipping, Eq. (11.46), on the whole gradient vector.

`fit(...)` forward pass, backpropagation through time and the update, `epochs` times over all windows in mini-batches of `batch_size` windows.

Keras: the knobs you will actually use

- `SimpleRNN` → `LSTM` → `GRU`: change one word; the shapes of input and output are the same.
- `return_sequences=True` on every layer but the last to *stack* recurrent layers; `return_state=True` to also get the final state (for the LSTM both $h^{(T)}$ and $c^{(T)}$).
- `Input(shape=(T, n_x))` with a fixed T when all windows have the same length (faster); `None` when they do not.
- Variable-length sequences in one batch: pad, and put a `layers.Masking(mask_value=0.0)` first so that padded steps are skipped.
- Classification from the last state: `Dense(n_classes)` with `SparseCategoricalCrossentropy(from_logits=True)` – the softmax and the cross entropy of week 41 in one object.
- `LSTM(..., unit_forget_bias=True)` is the default: the forget gate starts at $\sigma(1) \approx 0.73$ (Part 6).

PyTorch: the same model

```
import torch, torch.nn as nn

class Forecaster(nn.Module):
    def __init__(self, n_h=24):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=n_h, batch_first=True) # Eq. (11.4), tanh
        self.fc = nn.Linear(n_h, 1) # V h_t + c

    def forward(self, x): # x: (N, T, 1)
        H, h_T = self.rnn(x) # H: (N, T, n_h) = every h_t; h_T: (1, N, n_h)
        return self.fc(H) # (N, T, 1): one prediction per step

model = Forecaster()
print(sum(p.numel() for p in model.parameters())) # 673 (two bias vectors, see below)
```

PyTorch separates the *definition* of the network (a class with a `forward`) from its *training* (a loop you write). Keras hides the loop in `fit`; PyTorch shows it – which is why we use it when we want to see the gradients.

PyTorch: what each line does

`nn.RNN(input_size, hidden_size, batch_first=True)` Eq. (11.4) with tanh (`nonlinearity="relu"` is the other option). `batch_first=True` makes the input (N, T, n_x). `num_layers=2` stacks two layers.

`self.rnn(x)` returns a pair `H` of shape (N, T, n_h) with all hidden states, and `h_T` of shape (`num_layers`, N, n_h) with the last one. You choose: all of `H` for sequence-to-sequence, `H[:, -1, :]` for sequence-to-label.

the parameters `weight_ih_l0` is U ($n_h \times n_x$), `weight_hh_l0` is W, and there are *two* biases `bias_ih_l0` and `bias_hh_l0` whose sum is our b – hence $673 = 649 + 24$.

`nn.Linear(n_h, 1)` $\mathsf{V}\mathsf{h} + \mathsf{c}$ on the last axis, exactly like Keras's `Dense`.

`nn.LSTM`, `nn.GRU` same calls, same shapes; `nn.LSTM` returns `(H, (h_T, c_T))`, with the cell state as well.

PyTorch: the training loop, line by line

```
opt = torch.optim.Adam(model.parameters(), lr=5e-3)
lossfn = nn.MSELoss()
for epoch in range(300):
    for idx in torch.randperm(len(X_train)).split(4): # shuffle windows, batches of 4
        opt.zero_grad() # forget last batch's gradients
        loss = lossfn(model(X_train[idx]), Y_train[idx]) # forward pass, Eq. (11.5)
        loss.backward() # BPTT, Eqs. (11.20)-(11.22)
        gnorm = nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Eq. (11.46); returns ||g||
        opt.step() # Adam update, week 38
```

- `randperm(...).split(4)`: the *windows* are shuffled, the time order inside each window is not.
- `zero_grad()`: PyTorch *accumulates* gradients into `.grad`, so each batch starts by clearing them.
- `loss.backward()`: automatic differentiation through the unrolled graph – Part 4 says what it computes.
- `clip_grad_norm_` rescales the whole gradient in place if its norm exceeds 1 and returns the norm *before* clipping – which is how we will look at it.

Counting parameters in both libraries

```
cell Keras PyTorch formula n_h(n_x+n_h+1) n_x=1, n_h=24
simple RNN 624 648 624
LSTM 2496 2592 2496 (4 gates: four times the simple RNN)
GRU 1944 1944 1872 (3 gates: three times)
```

- Keras's `SimpleRNN` and `LSTM` print exactly Eq. (11.4) and 4×Eq. (11.4). Use this as a check: if the LSTM is not four times the simple RNN, something is wrong.
- PyTorch is larger by n_h per gate: the second bias vector. The model is the same, only the parameterisation is redundant.
- Keras's `GRU` uses `reset_after=True` (the cuDNN convention) which also carries two biases – hence 1944 in both.

Same weights, same outputs: the conventions (Section 11.8.3)

```
rnn = nn.RNN(n_x, n_h, batch_first=True) # PyTorch, random weights
U = rnn.weight_ih_l0.detach().numpy() # (n_h, n_x)
W = rnn.weight_hh_l0.detach().numpy() # (n_h, n_h)
b = (rnn.bias_ih_l0 + rnn.bias_hh_l0).detach().numpy() # one bias in Eq. (11.4)
k = keras.Sequential([keras.Input(shape=(T, n_x)), layers.SimpleRNN(n_h, return_sequences=True)])
k.layers[0].set_weights([U.T, W.T, b]) # Keras stores the transposes
print(np.max(np.abs(rnn(X)[0].detach().numpy() - k(X.numpy()).numpy()))) # 1.2e-07
```

Three conventions differ: PyTorch has two biases where Eq. (11.4) has one; Keras stores U and W transposed (`kernel` is (n_x, n_h)); and both libraries concatenate the LSTM gates in the order $(\mathrm{i}, \mathrm{f}, \mathrm{g}, \mathrm{o})$ where Eqs. (11.49)–(11.52) introduce them as $(\mathrm{f}, \mathrm{i}, \mathrm{g}, \mathrm{o})$. With those reconciled the hidden states agree to 10^{-7} – single precision – for the same inputs.

Pause and think 3: shapes

Question

A batch of 32 windows of length 20 with one feature is fed to `nn.RNN(1, 16, batch_first=True)`. What are the shapes of the two outputs? Which slice gives the input to a classifier that sees the whole window? And what does Keras's `SimpleRNN(16)` return for the same batch with `return_sequences=False` and `True`?

Pause and think 3: shapes

Question

A batch of 32 windows of length 20 with one feature is fed to `nn.RNN(1, 16, batch_first=True)`. What are the shapes of the two outputs? Which slice gives the input to a classifier that sees the whole window? And what does Keras's `SimpleRNN(16)` return for the same batch with `return_sequences=False` and `True`?

Answer

`H` has shape $(32, 20, 16)$ – every hidden state of every window – and `h_T` has shape $(1, 32, 16)$: one layer, the last state of each window. The classifier reads `H[:, -1, :]`, of shape $(32, 16)$, which is the same tensor as `h_T[0]`. Keras returns $(32, 16)$ with `return_sequences=False` (the last state) and $(32, 20, 16)$ with `True` (all of them). Forecasting one step from a window of 20 uses the first; forecasting at every step uses the second and a `Dense(1)` that acts on the last axis.

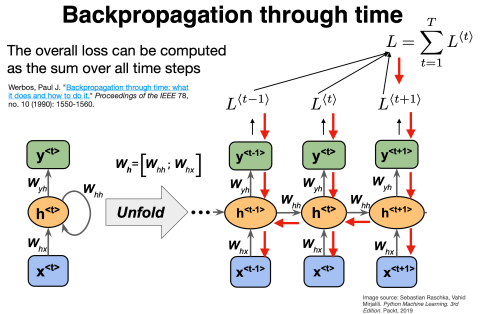
Training: unroll, and backpropagate (Section 11.4)

The idea

A recurrent network run for T steps *is* a feed-forward network of depth T in which every layer shares the same weights. Backpropagation (week 41) applies unchanged; the only novelty is that the gradients from every step must be *summed* into the same parameter arrays.

In the time domain

1. The forward pass builds a *stack* of the activities at each time step.
2. The backward pass peels the stack to compute the error derivatives at each time step.
3. The derivatives at all times are added together for each weight.



The red arrows are the backward pass; the loss is a sum over time (Werbos 1990).

The output layer and the last hidden state

Differentiating the cost (11.5) gives, for each t, Eq. (11.18),

$$(\nabla_{\mathsf{o}^{(t)}}\mathcal{L})_i = \frac{\partial \mathcal{L}}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}},$$

which for the quadratic cost and a linear σ_y is simply $(\hat{\mathsf{y}}^{(t)} - \mathsf{y}^{(t)})/T$ – the output error of week 41, divided by the number of steps.

At $t = T$ the state influences the cost *only* through the output at that step, Eq. (11.19):

$$\nabla_{\mathsf{h}^{(T)}}\mathcal{L} = \mathsf{V}^{\mathsf{T}} \nabla_{\mathsf{o}^{(T)}}\mathcal{L}.$$

This is where the backward pass starts: at the last step, with nothing arriving from the future.

Earlier hidden states have two routes to the cost

For $t < T$ the state $\mathsf{h}^{(t)}$ reaches the cost through the output at step t *and* through the state at step $t+1$. The chain rule gives two terms, Eq. (11.20),

$$\boxed{\nabla_{\mathsf{h}^{(t)}}\mathcal{L} = \left(\frac{\partial \mathsf{h}^{(t+1)}}{\partial \mathsf{h}^{(t)}}\right)^{\mathsf{T}} \nabla_{\mathsf{h}^{(t+1)}}\mathcal{L} + \left(\frac{\partial \mathsf{o}^{(t)}}{\partial \mathsf{h}^{(t)}}\right)^{\mathsf{T}} \nabla_{\mathsf{o}^{(t)}}\mathcal{L}}$$

and with Eq. (11.4) the two Jacobians are explicit, Eq. (11.21):

$$\nabla_{\mathsf{h}^{(t)}}\mathcal{L} = \mathsf{W}^{\mathsf{T}} \operatorname{diag}\left(\sigma_h'(\mathsf{a}^{(t+1)})\right) \nabla_{\mathsf{h}^{(t+1)}}\mathcal{L} + \mathsf{V}^{\mathsf{T}} \nabla_{\mathsf{o}^{(t)}}\mathcal{L}.$$

Equations (11.19) and (11.21), run backwards from $t = T$ to $t = 1$, are the whole of backpropagation through time.

Compare week 41

The first term

is the δ-recursion of Eq. (8.64): W^T times the error of the next layer times the derivative of the activation – with "next layer" meaning "next time step" and the same W at every step.

The second term

is new: every layer of the unrolled network has its own output and its own piece of the cost.

Which steps are supervised? Eq. (11.23) and Figure 11.2

Write the cost with a weight for every step,

$$\mathcal{L} = \sum_{t=1}^{T} \alpha_t \, L^{(t)}, \qquad \alpha_t \geq 0.$$

per-step supervision	$\alpha_t = 1$ (sum) or $1/T$ (mean)
final-time supervision	$\alpha_T = 1$, all earlier $\alpha_t = 0$
partially labelled	$\alpha_t = 0$ where a target is missing

The oscillator and the sine wave are per-step; MNIST, the spoken digits and the adding problem are final-time. An unlabelled early step still influences a later labelled one *through the hidden state* – which is where its gradient comes from.

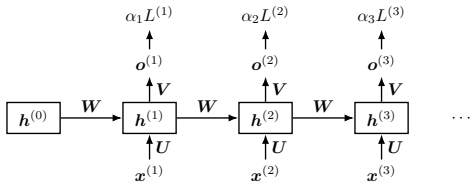


Figure 11.2: each loss $\alpha_t L^{(t)}$ hangs off its own output; the hidden states chain the steps together.

The error supplied by the readout, Eq. (11.24)

Let $\mathrm{e}^{(t)} = \nabla_{\mathrm{o}^{(t)}}(\alpha_t L^{(t)})$ be the error at the output of step t. Since $\mathrm{o}^{(t)} = \mathrm{V}\mathrm{h}^{(t)} + \mathrm{c}$, the *direct* signal at the hidden state is $\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(t)}$.

$$\text{regression:}\quad \hat{\mathrm{y}}^{(t)} = \mathrm{o}^{(t)},\ L^{(t)} = \tfrac{1}{2}\|\mathrm{o}^{(t)} - \mathrm{y}^{(t)}\|^2 \quad\Longrightarrow\quad \mathrm{e}^{(t)} = \alpha_t\big(\mathrm{o}^{(t)} - \mathrm{y}^{(t)}\big),$$

$$\text{classification:}\quad p_i^{(t)} = \frac{e^{o_i^{(t)}}}{\sum_j e^{o_j^{(t)}}},\ L^{(t)} = -\sum_i y_i^{(t)} \log p_i^{(t)} \quad\Longrightarrow\quad \mathrm{e}^{(t)} = \alpha_t\big(\mathrm{p}^{(t)} - \mathrm{y}^{(t)}\big).$$

The same "prediction minus target" as in weeks 39 and 41 (softmax with the cross entropy, Chapter 5): rewrite $L^{(t)} = -\mathrm{y}^{(t)\mathsf{T}}\mathrm{o}^{(t)} + \log\sum_j e^{o_j^{(t)}}$ and differentiate. With $\alpha_t = 1/T$ this is the $(\hat{\mathrm{y}} - \mathrm{y})/T$ of Eq. (11.18).

The recursion in the pre-activation error, Eqs. (11.25)–(11.27)

Perturb $\mathrm{h}^{(t-1)}$ with $\mathrm{x}^{(t)}$ and the parameters fixed: $\mathrm{da}^{(t)} = \mathrm{W}\mathrm{dh}^{(t-1)}$, $\mathrm{dh}^{(t)} = \mathrm{D}^{(t)}\mathrm{da}^{(t)}$ with $\mathrm{D}^{(t)} = \mathrm{diag}(\sigma_h'(\mathrm{a}^{(t)}))$, so the **step Jacobian** is

$$\mathrm{J}^{(t)} = \frac{\partial \mathrm{h}^{(t)}}{\partial \mathrm{h}^{(t-1)}} = \mathrm{D}^{(t)}\mathrm{W} \qquad (\text{for } \tanh\colon\ \mathrm{D}^{(t)} = \mathrm{diag}(1 - h_i^{(t)2})).$$

Call $\mathrm{q}^{(t)} = \mathrm{V}^{\mathsf{T}}\mathrm{e}^{(t)}$ the direct signal and $\mathrm{g}^{(t)} = \nabla_{\mathrm{h}^{(t)}}\mathcal{L}$ the total one. Then Eq. (11.20) reads $\mathrm{g}^{(\tau)} = \mathrm{q}^{(\tau)}$, $\mathrm{g}^{(t)} = \mathrm{q}^{(t)} + \mathrm{J}^{(t+1)\mathsf{T}}\mathrm{g}^{(t+1)}$, and in the pre-activation error $\boldsymbol{\delta}^{(t)} = \mathrm{D}^{(t)}\mathrm{g}^{(t)}$:

$$\boxed{\boldsymbol{\delta}^{(t)} = \left(\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(t)} + \mathrm{W}^{\mathsf{T}}\boldsymbol{\delta}^{(t+1)}\right) \odot \sigma_h'(\mathrm{a}^{(t)}), \qquad \boldsymbol{\delta}^{(\tau+1)} = 0.}$$

Read it

$\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(t)}$ arrives from the *current* output; $\mathrm{W}^{\mathsf{T}}\boldsymbol{\delta}^{(t+1)}$ returns from the *next* transition. If $\alpha_t = 0$ only the first term vanishes. This is the δ-recursion of Eq. (8.64) with "next layer" = "next step", and it is the single line `da = (V.T @ e + W.T @ da_next) * (1 - h**2)` of every hand-written BPTT – the line the libraries execute for us.

Three steps written out, Eq. (11.28)

For $T = 3$, unrolling the box:

$$\begin{aligned}
\boldsymbol{\delta}^{(3)} &= \mathrm{D}^{(3)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(3)}, \\
\boldsymbol{\delta}^{(2)} &= \mathrm{D}^{(2)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(2)} + \mathrm{D}^{(2)}\mathrm{W}^{\mathsf{T}}\mathrm{D}^{(3)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(3)}, \\
\boldsymbol{\delta}^{(1)} &= \mathrm{D}^{(1)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(1)} + \mathrm{D}^{(1)}\mathrm{W}^{\mathsf{T}}\mathrm{D}^{(2)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(2)} + \mathrm{D}^{(1)}\mathrm{W}^{\mathsf{T}}\mathrm{D}^{(2)}\mathrm{W}^{\mathsf{T}}\mathrm{D}^{(3)}\mathrm{V}^{\mathsf{T}}\mathrm{e}^{(3)}.
\end{aligned}$$

What to see

- The last term of $\boldsymbol{\delta}^{(1)}$ is the complete path from the third loss back to the first pre-activation: two factors $\mathrm{W}^{\mathsf{T}}\mathrm{D}$ – one per step crossed. For a final-time cost it is the *only* term in $\boldsymbol{\delta}^{(1)}$.
- Every factor $\mathrm{W}^{\mathsf{T}}\mathrm{D}^{(t)}$ is the transpose of a step Jacobian: the product that Theorem 11.1 will bound.
- Then $\nabla_{\mathrm{W}}\mathcal{L} = \boldsymbol{\delta}^{(1)}\mathrm{h}^{(0)\mathsf{T}} + \boldsymbol{\delta}^{(2)}\mathrm{h}^{(1)\mathsf{T}} + \boldsymbol{\delta}^{(3)}\mathrm{h}^{(2)\mathsf{T}}$.

The parameters: every step contributes, Eq. (11.22)

Because the same matrices act at every step, each accumulates a contribution from every step. With $\boldsymbol{\delta}^{(t)} = \operatorname{diag}(\sigma_h'(\mathsf{a}^{(t)}))\,\nabla_{\mathsf{h}^{(t)}}\mathcal{L}$,

$$\nabla_{\mathsf{c}}\mathcal{L} = \sum_t \nabla_{\mathsf{o}^{(t)}}\mathcal{L}, \qquad \nabla_{\mathsf{V}}\mathcal{L} = \sum_t (\nabla_{\mathsf{o}^{(t)}}\mathcal{L})\,\mathsf{h}^{(t)\mathsf{T}},$$

$$\nabla_{\mathsf{b}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}, \qquad \nabla_{\mathsf{W}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathsf{h}^{(t-1)\mathsf{T}}, \qquad \nabla_{\mathsf{U}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathsf{x}^{(t)\mathsf{T}}.$$

Each term is the week-41 formula "error times the input of that layer", Eqs. (8.67)–(8.68); the sum over t is the weight sharing.

Why a matrix gradient is a sum of outer products, Eqs. (11.29)–(11.31)

At step t perturb only the *local copy* of W, holding $\mathrm{h}^{(t-1)}$ fixed: $\mathrm{da}^{(t)} = (\mathrm{dW})\mathrm{h}^{(t-1)}$, so

$$\mathrm{d}\mathcal{L}\big|_t = \boldsymbol{\delta}^{(t)\mathsf{T}}(\mathrm{dW})\mathrm{h}^{(t-1)} = \sum_{i,j} \delta_i^{(t)} h_j^{(t-1)}\,\mathrm{d}W_{ij}.$$

Every occurrence of W receives the same perturbation, hence

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_{t=1}^{T} \delta_i^{(t)} h_j^{(t-1)}, \qquad \nabla_{\mathrm{W}}\mathcal{L} = \sum_{t=1}^{T} \boldsymbol{\delta}^{(t)}\mathrm{h}^{(t-1)\mathsf{T}}.}$$

The complete table, with shapes

$\nabla_{\mathrm{U}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathrm{x}^{(t)\mathsf{T}}$	$n_h \times n_x$
$\nabla_{\mathrm{W}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathrm{h}^{(t-1)\mathsf{T}}$	$n_h \times n_h$
$\nabla_{\mathrm{b}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}$	n_h
$\nabla_{\mathrm{V}}\mathcal{L} = \sum_t \mathrm{e}^{(t)}\mathrm{h}^{(t)\mathsf{T}}$	$n_y \times n_h$
$\nabla_{\mathrm{c}}\mathcal{L} = \sum_t \mathrm{e}^{(t)}$	n_y

Two more for free

The same reverse pass gives $\nabla_{\mathrm{x}^{(t)}}\mathcal{L} = \mathrm{U}^{\mathsf{T}}\boldsymbol{\delta}^{(t)}$ (the input gradient – adversarial examples, saliency) and $\nabla_{\mathrm{h}^{(0)}}\mathcal{L} = \mathrm{W}^{\mathsf{T}}\boldsymbol{\delta}^{(1)}$, which is what an *encoder* supplying the initial state receives.

The path-sum view, Eqs. (11.32)–(11.34)

Let $\mathsf{P}^{(t:k)} = \partial \mathsf{h}^{(t)} / \partial \mathsf{h}^{(k)}$ for $t \geq k$:

$$\mathsf{P}^{(t:k)} = \mathsf{J}^{(t)} \mathsf{J}^{(t-1)} \cdots \mathsf{J}^{(k+1)}, \qquad \mathsf{P}^{(k:k)} = \mathsf{I}, \qquad \mathsf{g}^{(k)} = \sum_{t=k}^{T} \mathsf{P}^{(t:k)\mathsf{T}} \mathsf{q}^{(t)}.$$

For a parameter θ of the recurrent update, with $\mathsf{B}^{(k)} = \partial \mathsf{h}^{(k)} / \partial \theta$ at fixed $\mathsf{h}^{(k-1)}$,

$$\boxed{\nabla_\theta \mathcal{L} = \sum_{t=1}^{T} \sum_{k=1}^{t} \mathsf{B}^{(k)\mathsf{T}} \mathsf{P}^{(t:k)\mathsf{T}} \mathsf{q}^{(t)} = \sum_{k=1}^{T} \mathsf{B}^{(k)\mathsf{T}} \mathsf{g}^{(k)}.}$$

Three views of one gradient

- **Path sum** (left): loss t reaches parameter use k through the ordered product of $t-k$ Jacobians – the view that explains vanishing and exploding gradients.
- **Backward recursion** (right): the reverse sweep evaluates *all* these sums without enumerating a single path – the view the code implements, $\mathcal{O}(T)$ instead of $\mathcal{O}(T^2)$.
- **Outer products** (previous frame): what to accumulate at each step.

A worked example, 1: two steps and one final target (Section 11.4.5)

A scalar network: $h_t = \tanh(ux_t + wh_{t-1})$, $o_t = vh_t$, $\mathcal{L} = \frac{1}{2}(o_2 - y)^2$ – final-time supervision, $\alpha_1 = 0$.

$$u = v = 1, \qquad w = 0.5, \qquad h_0 = 0, \qquad (x_1, x_2) = (1, 0), \qquad y = 0.5.$$

t	x_t	a_t	h_t	$1 - h_t^2$
1	1	1.000000	0.761594	0.419974
2	0	0.380797	0.363399	0.867941

The residual is $e_2 = o_2 - y = -0.136601$ and $\mathcal{L} = 0.00932985$.

Why this example

The second input is zero: it contributes nothing directly. *All* information about x_1 must reach the prediction through the recurrent weight w – a delayed-copy task in miniature.

A worked example, 2: tracing the error back to the first input

With $e_1 = 0$ and $\delta_3 = 0$, the boxed recursion gives

$$\delta_2 = (1 - h_2^2)\, v\, e_2 = -0.118561, \qquad \delta_1 = (1 - h_1^2)\, w\, \delta_2 = -0.024896.$$

derivative	contributions	value
$\partial \mathcal{L}/\partial w$	$\delta_1 h_0 + \delta_2 h_1$	-0.090295
$\partial \mathcal{L}/\partial u$	$\delta_1 x_1 + \delta_2 x_2$	-0.024896
$\partial \mathcal{L}/\partial v$	$e_2 h_2$	-0.049641
$\partial \mathcal{L}/\partial x_1$	$u\, \delta_1$	-0.024896

A centred finite difference in w with $\epsilon = 10^{-6}$ gives -0.09029549. A reverse pass that *stopped at* h_1 would give zero for the first-step contribution to $\partial \mathcal{L}/\partial u$: the whole gradient of u here comes from the step before the target.

The same example with `torch.autograd`

```
import torch
u, v, w = (torch.tensor(a, requires_grad=True)
           for a in (1.0, 1.0, 0.5))
x1 = torch.tensor(1.0, requires_grad=True)
x2, h0, y = torch.tensor(0.0), 0.0, 0.5

h1 = torch.tanh(u * x1 + w * h0)
h2 = torch.tanh(u * x2 + w * h1)
L = 0.5 * (v * h2 - y) ** 2
L.backward() # BPTT, two steps
print(h1.item(), h2.item(), L.item())
print(w.grad, u.grad, v.grad, x1.grad)
```

```
0.761594 0.363400 0.009330
-0.090295 -0.024896 -0.049641 -0.024896
```

What the check establishes

Autograd reproduces the hand calculation to six digits, including the gradient with respect to the *input* x_1 – the libraries do exactly the reverse pass of the last frames, nothing more.

How to design a gradient check

Short deterministic sequence, double precision; a *non-zero* h_0 so that the first outer product $\boldsymbol{\delta}^{(1)}\mathrm{h}^{(0)\mathsf{T}}$ is exercised; all of $\mathsf{U}, \mathsf{W}, \mathsf{V}, \mathsf{b}, \mathsf{c}$ and the initial-state gradient; the same loss reduction on both sides; clipping off. A test with $T = 1$ and $h_0 = 0$ cannot expose a missing temporal path.

The algorithm, and what it costs (Section 11.4.6, Eq. (11.35))

One forward pass, one reverse pass

1. For $t = 1, \dots, T$: compute $\mathsf{a}^{(t)}, \mathsf{h}^{(t)}, \mathsf{o}^{(t)}$, accumulate $\alpha_t L^{(t)}$, cache what the reverse pass needs.
2. Zero the gradient accumulators; $\boldsymbol{\delta}^{(T+1)} = 0$.
3. For $t = T, \dots, 1$: $\mathsf{e}^{(t)}$, then the boxed $\boldsymbol{\delta}^{(t)}$.
4. At the same step add the outer products to the matrix gradients and the error vectors to the bias gradients.
5. After the loop: one optimizer step. Never change W or V *inside* the reverse loop – that would differentiate a different computation from the one that produced the loss.

Cost

Work per sequence, forward and backward:

$$\mathcal{O}\big(T\,(n_h n_x + n_h^2 + n_y n_h)\big),$$

storage $\mathcal{O}(Tn_h)$ for the hidden states (plus the cached activations).

The sequential wall

Within a step the matrix products parallelize; across steps $\mathsf{h}^{(t)}$ must exist before $\mathsf{h}^{(t+1)}$, and batching independent sequences does not remove that. This is the limitation that attention (Chapter 13) was invented to escape. *Checkpointing* stores a few states and recomputes the rest in the backward pass: exact gradient, compute traded for memory – unlike truncation, which changes the gradient.

BPTT for the Euler oscillator is the discrete adjoint (Section 11.4.7)

Take the linear network of Eq. (11.17), $\mathsf{s}_{i+1} = \mathsf{A}(\eta)\mathsf{s}_i + \mathsf{B}F_i$, and suppose *only the final position* is observed: $\mathcal{L} = \frac{1}{2}(\mathsf{C}\mathsf{s}_N - x_N^{\mathrm{obs}})^2$ with $\mathsf{C} = (1, 0)$. The backward recursion (11.20) with $\mathsf{D} = \mathsf{I}$ and $\mathsf{J} = \mathsf{A}$ is

$$\boldsymbol{\lambda}_N = \mathsf{C}^{\mathsf{T}}\big(\mathsf{C}\mathsf{s}_N - x_N^{\mathrm{obs}}\big), \qquad \boldsymbol{\lambda}_i = \mathsf{A}^{\mathsf{T}}\boldsymbol{\lambda}_{i+1}, \tag{11.36}$$

and since $\partial\mathsf{A}/\partial\eta$ has the single entry $-\Delta t$ in position $(2, 2)$,

$$\boxed{\frac{\partial \mathcal{L}}{\partial \eta} = \sum_{i=0}^{N-1} \boldsymbol{\lambda}_{i+1}^{\mathsf{T}} \frac{\partial \mathsf{A}}{\partial \eta}\mathsf{s}_i = -\Delta t \sum_{i=0}^{N-1} \lambda_{i+1,v}\, v_i.} \tag{11.37}$$

Read it

A single residual at the end trains the damping through *every* earlier velocity update: this is the adjoint method of numerical analysis, and it is backpropagation through time for the Euler model – the same algorithm under two names. (Exercise 15 of the chapter: recover η from one final observation.)

The backward pass is linear

There is an asymmetry between the two passes that is easy to miss.

Forward

Non-linear: σ_h squashes, which keeps the activities bounded and is what prevents them from exploding.

Backward

Completely linear in the incoming gradient: Eq. (11.20) is a matrix acting on $\nabla_{\mathbf{h}^{(t+1)}} \mathcal{L}$, the matrix fixed by the forward pass. Double the error at the final step and every derivative doubles.

Nothing bounds the backward pass, which is precisely why it can explode when the forward pass cannot.

What `loss.backward()` computes

We do not write Eqs. (11.19)–(11.22) ourselves: both libraries record the unrolled graph during the forward pass and differentiate it backwards – reverse-mode automatic differentiation (week 38). Two consequences:

- the gradient is *exactly* the recursion above, to rounding;
- it is a fixed linear map of the output error, which we can test:

```
rnn, fc = nn.RNN(2, 6, batch_first=True).double(), nn.Linear(6, 1).double()
for scale in (1.0, 2.0): # output error e, then 2e
    for q in params: q.grad = None
    loss = 0.5 * ((fc(rnn(X)[0]) - target(scale)) ** 2).sum()
    loss.backward() # BPTT
    grads.append([q.grad.clone() for q in params])
# max |g(2e) - 2 g(e)| / max|g(2e)| = 4.3e-16 (float64: exact)
```

In Keras the same thing happens inside `model.fit`; to see it, use `tf.GradientTape()` around the forward pass and call `tape.gradient(loss, model.trainable_variables)`.

Pause and think 4: what travels backwards

Question

Suppose the cost is non-zero only at the last step ($L^{(t)} = 0$ for $t < T$, the sequence-to-label case, `H[:, -1, :]`). Which term of Eq. (11.20) survives at the earlier steps – and how many times has W^{T} been applied to the error by the time it reaches $t = 1$?

Pause and think 4: what travels backwards

Question

Suppose the cost is non-zero only at the last step ($L^{(t)} = 0$ for $t < T$, the sequence-to-label case, `H[:, -1, :]`). Which term of Eq. (11.20) survives at the earlier steps – and how many times has $W^\top$ been applied to the error by the time it reaches $t = 1$?

Answer

With a cost at the last step only, the second term $V^\top \nabla_{o^{(t)}} \mathcal{L}$ is zero for every $t < T$, and the gradient at $t = 1$ is $\prod_{k=2}^{T} W^\top \operatorname{diag}(\sigma_h'(a^{(k)}))$ applied to the single output error: $T - 1$ factors of $W^\top$, each multiplied by a $\tanh'$ that is at most one. That product is the subject of Part 5. Everything that follows is a statement about this product – and `loss.backward()` computes it faithfully, which is the problem, not the cure.

A product of Jacobians (Section 11.5)

Iterating Eq. (11.21) from T back to t produces a *product* of Jacobians, one factor per step. It is this product, and not anything about the architecture as such, that makes recurrent networks hard to train.

Theorem 11.1 (vanishing and exploding gradients)

Let $\mathrm{h}^{(t)} = \sigma_h(\mathrm{U}\mathrm{x}^{(t)} + \mathrm{W}\mathrm{h}^{(t-1)} + \mathrm{b})$ with $|\sigma_h'(z)| \le \mu$. Then for $t < T$, Eq. (11.38),

$$\frac{\partial \mathrm{h}^{(T)}}{\partial \mathrm{h}^{(t)}} = \prod_{k=t+1}^{T} \operatorname{diag}\!\left(\sigma_h'(\mathrm{a}^{(k)})\right) \mathrm{W},$$

and its spectral norm obeys, Eq. (11.39),

$$\left\| \frac{\partial \mathrm{h}^{(T)}}{\partial \mathrm{h}^{(t)}} \right\|_2 \le \left(\mu\, \sigma_{\max}(\mathrm{W})\right)^{T-t}.$$

If $\mu\sigma_{\max} < 1$ the contribution of step t decays *exponentially* in the lag $T - t$; if $\mu\sigma_{\max} > 1$ it may grow exponentially.

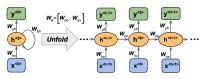
The proof in one line, and the picture

One chain-rule factor per step; the spectral norm is submultiplicative; each factor has norm at most

$$\| \operatorname{diag}(\sigma_h') \|_2 \, \|\mathrm{W}\|_2 \leq \mu \, \sigma_{\max}(\mathrm{W}).$$

The bound is *sufficient* for vanishing, not necessary: it uses $\sigma_{\max}(\mathrm{W})$, and for a random (non-normal) W this is larger than $\rho(\mathrm{W})$. We will see the gap in the measurement.

Backpropagation through time



Werbos, Paul J. "Backpropagation through time: what it does and how to do it." *Proceedings of the IEEE* 78, no. 10 (1990): 1550-1560.

$$L = \sum_{t=1}^{T} L^{(t)} \qquad \frac{\partial L^{(t)}}{\partial \mathbf{W}_{hh}} = \frac{\partial L^{(t)}}{\partial y^{(t)}} \cdot \frac{\partial y^{(t)}}{\partial \mathbf{h}^{(t)}} \cdot \left(\sum_{k=1}^{t} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} \cdot \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{W}_{hh}} \right)$$

computed as a multiplication of adjacent time steps:

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(i-1)}}$$

This is very problematic:
Vanishing/Exploding gradient problem!

What the activation function decides

Why tanh

For tanh, $\mu = 1$, attained only at the origin.

Why not the logistic function

$\mu = 1/4$: a logistic recurrent network loses a factor of at least four per step *regardless of the weights* – two bits of gradient per step, 10^{35} over sixty steps.

Why not ReLU

$\mu = 1$ too, but the derivative is 0 or 1 rather than a number approaching 1 smoothly, and a ReLU recurrent network is prone to explosion. This is why the rectified family, dominant everywhere else in the course, is largely absent from recurrent architectures.

Corollary 11.2: the memory horizon

If $\mu\sigma_{\max}(W) < 1$, define Eq. (11.40),

$$\tau = \frac{-1}{\log(\mu\,\sigma_{\max}(W))} > 0,$$

so that the bound reads $e^{-(T-t)/\tau}$: the influence of step t on step T decays with a characteristic scale of τ steps and falls below ε once $T - t > \tau \log(1/\varepsilon)$.

A design parameter

τ converts the theorem from a qualitative warning into a number: *it is the length of sequence the network can actually use*. We measure it two frames on, and the answer is smaller than one expects.

The linear case: eigenmodes, Eq. (11.41) (Section 11.5.1)

Suppress inputs and biases and take σ_h linear: $\mathrm{h}^{(t)} = \mathrm{W}^t \mathrm{h}^{(0)}$. If $\mathrm{W}\mathrm{v}_i = \lambda_i \mathrm{v}_i$ and $\mathrm{h}^{(0)} = \sum_i a_i \mathrm{v}_i$,

$$\boxed{\mathrm{W}^t \mathrm{h}^{(0)} = \sum_i a_i \lambda_i^t \mathrm{v}_i.}$$

Modes with $|\lambda_i| < 1$ decay, with $|\lambda_i| > 1$ grow, with $|\lambda_i| = 1$ oscillate; the mode with the largest $|\lambda_i|$ eventually dominates. The backward map over a lag d is $(\mathrm{W}^{\mathsf{T}})^d$, with the *same* singular values as W^d: whatever the forward pass does to a perturbation, the backward pass does to a loss signal.

Small differences in gain compound with delay

A scalar transition multiplies a perturbation by r per step: $0.9^{100} = 2.7 \times 10^{-5}$, $1.1^{100} = 1.4 \times 10^4$. For $0 < r < 1$, $r^d = e^{-d/\tau}$ with the *decay time* $\tau = -1/\log r$ – the memory horizon of Corollary 11.2 in its simplest form ($\tau = 9.5$ steps at $r = 0.9$, 99.5 at $r = 0.99$).

A small gradient can hide a long-range dependency, Eq. (11.42)

The delayed-copy task

$x_1 = s \in \{-1, +1\}$, all later inputs zero, final target $y_T = s$: the network must preserve *one bit* for $T - 1$ steps. The training signal at the step that saw the bit is

$$\nabla_{\mathsf{h}^{(1)}} L^{(T)} = \mathsf{P}^{(T:1)\mathsf{T}} \nabla_{\mathsf{h}^{(T)}} L^{(T)}, \qquad \|\mathsf{P}^{(T:1)}\|_2 \le \prod_{j=2}^{T} \|\mathsf{J}^{(j)}\|_2 \le \rho^{\,T-1} \quad \text{if } \|\mathsf{J}^{(j)}\|_2 \le \rho < 1.$$

The trap

A *sizeable* final error can produce a *tiny* gradient at the step that mattered – and the total parameter-gradient norm, dominated by recent steps, will not show it. The network then fits whatever short-range statistics exist and ignores the bit. This is exactly what the adding problem at $T = 30$ shows at the end of the lecture; the per-lag measurement of `week45rnn_gradients.pdf` is how to see it coming.

Saturation can shorten memory even when W is large

For a scalar tanh unit $J^{(t)} = w\,(1 - h^{(t)2})$; for a vector state

$$\|\mathrm{J}^{(t)}\|_2 \le \|\mathrm{D}^{(t)}\|_2\,\|\mathrm{W}\|_2, \qquad \mathrm{D}^{(t)} = \operatorname{diag}\big(1 - h_i^{(t)2}\big).$$

Numbers

At $a = 0$ the derivative of tanh is 1; at $a = 2$ it is 0.071. Even $w = 2$ gives a local gain of only 0.14 at that operating point. A logistic hidden unit has derivative at most $1/4$ *anywhere*.

Consequence

Large recurrent weights push units into saturation, where the gradient dies *faster*. The scale of the inputs and the biases matters as much as the scale of W: this is why the measurement of Theorem 11.1 was done with small inputs, and why the diagnostic table later asks for the distribution of $1 - h_i^{(t)2}$.

Exploding gradients are directional, and stability can hide transient growth, Eq. (11.43)

Directional

For a general trajectory $P^{(t:k)} = D^{(t)} W D^{(t-1)} W \cdots D^{(k+1)} W$. Its largest singular value is the greatest *possible* amplification; the actual backward signal depends on its direction and may stay small. An upper bound above one *permits* growth – it does not establish that a particular gradient explodes (which is what the measured table showed: bound 10^{13}, measured 3×10^{-3} at $\rho = 0.9$).

Transient growth with stable eigenvalues

$$W = \begin{pmatrix} 0.9 & 5 \\ 0 & 0.9 \end{pmatrix}, \quad W^n = \begin{pmatrix} 0.9^n & 5n\,0.9^{n-1} \\ 0 & 0.9^n \end{pmatrix}.$$

Both eigenvalues are 0.9, yet the upper-right entry of W^{10} is $50 \cdot 0.9^9 = 19.4$, and $\|W^n\|_2$ peaks at 19.4 for $n = 9$ before decaying (3.3 at $n = 40$). A non-normal W amplifies first and decays later: the spectral radius is a poor *finite-horizon* diagnostic, and in a non-linear network the $D^{(t)}$ depend on the inputs on top of that.

Remembering an input competes with rejecting noise, Eq. (11.44)

Suppose the transition is *contractive* in the state for a fixed input,

$$\|f(\mathrm{h},\mathrm{x}) - f(\tilde{\mathrm{h}},\mathrm{x})\| \le \rho\,\|\mathrm{h} - \tilde{\mathrm{h}}\|, \qquad 0 \le \rho < 1.$$

Two histories that differ at step k but receive identical inputs afterwards obey $\|\mathrm{h}^{(t)} - \tilde{\mathrm{h}}^{(t)}\| \le \rho^{\,t-k}\|\mathrm{h}^{(k)} - \tilde{\mathrm{h}}^{(k)}\|$.

The dilemma

Contraction is what makes the network *forget its initial condition* and *reject noise* – the property that let the untrained `nn.RNN` at $\rho(\mathrm{W}) = 0.5$ wash out $\mathrm{h}^{(0)}$ in a dozen steps. It is the same property that erases the difference between two early cues that *should* lead to different predictions. A finite state in finite precision must keep useful distinctions alive while new observations keep arriving; more hidden units give more directions to do so, but do not guarantee that training finds a persistent one. The gated cells of the next part exist to make "keep this" a learnable decision.

Truncated BPTT, Eq. (11.45)

Propagating the gradient back only k steps drops the tail of the product; the error is bounded by

$$\frac{C\, e^{-k/\tau}}{e^{1/\tau} - 1},$$

so k a few multiples of τ is safe.

But read what this says: truncation is cheap precisely when the recurrence is *not* doing the thing it was introduced for.

Truncated BPTT: what a boundary removes

Suppose $T = 6$ and the chunks have length three. At the boundary we carry $\tilde{h}^{(3)} = \text{stopgrad}(h^{(3)})$: its *value* equals $h^{(3)}$, its derivative with respect to how that value was computed is zero.

$$\underbrace{h^{(1)}, h^{(2)}, h^{(3)}}_{\text{chunk 1}} \quad \Big| \quad \underbrace{h^{(4)}, h^{(5)}, h^{(6)}}_{\text{chunk 2}}$$

A loss at $t = 6$ still *uses* the information in $\tilde{h}^{(3)}$; it sends *no gradient* across the boundary to the parameter uses at $t = 1, 2, 3$. The uses at $t = 4, 5, 6$ receive theirs. Storage per update drops from T to k states; the gradient is no longer the gradient of the cost – by how much depends on the paths that were cut, Eq. (11.45).

Carrying, detaching or resetting the state (Section 11.5.4)

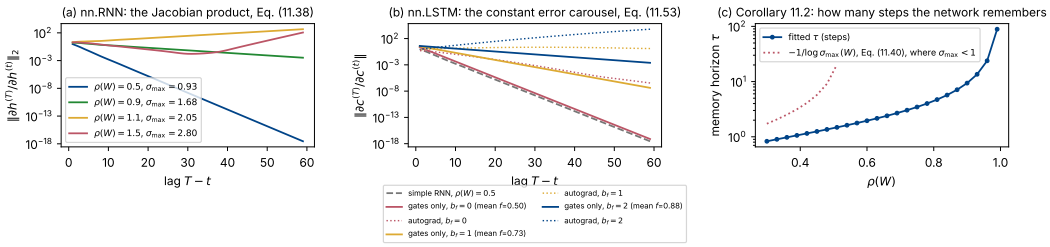
at a boundary	state	gradient to the past
carry with history	kept	kept
carry and detach	kept	cut
reset to $h^{(0)}$	reinitialized	cut

Carry a state only between consecutive chunks of the *same* trajectory; reset between independent examples, or a batch passes one sequence's history into another. For padded, variable-length sequences a zero loss weight removes the supervision at a padded step but does not freeze the state – stop at the true length or keep the last valid state.

```
# PyTorch: carry and detach between chunks of ONE sequence
h = None
for chunk, target in chunks: # (1, k, n_x) each
    out, h = model(chunk, h)
    loss = lossfn(out, target)
    opt.zero_grad(); loss.backward(); opt.step()
    h = h.detach() # value kept, graph cut
# reset between independent sequences: h = None

# Keras: a stateful layer keeps h between batches ...
layers.SimpleRNN(n_h, stateful=True,
                 batch_input_shape=(1, k, n_x))
model.reset_states() # ... until this call
```

Measured with `torch.autograd` on `nn.RNN` (`week45rnn_gradients.pdf`)



$n_h = 20$, $T = 60$, inputs small enough that $\tanh' \approx 1$, W scaled to a prescribed spectral radius; each curve is `torch.autograd.functional.jacobian` of $h^{(T)}$ with respect to $h^{(t)}$. (a) At $\rho(W) = 0.5$ the influence of the first step on the last has fallen by *eighteen orders of magnitude* over 59 steps. At $\rho = 1.1$ it has grown to 450. (b) is for Part 6. (c) The horizon τ fitted to the slope of (a).

The numbers behind the figure

```
rho(W) sigma_max mean max(tanh') ||dh_T/dh_1|| bound sigma_max^59 fitted tau
 0.50 0.933 1.0000 2.878e-18 1.705e-02 1.44
 0.90 1.680 1.0000 3.313e-03 1.963e+13 9.38
 1.10 2.053 1.0000 4.547e+02 2.721e+18 -10.63
 1.50 2.800 0.9980 1.152e+02 2.410e+26 -20.40

fitted memory horizon and the lag at which the influence has fallen by 1e-6:
  rho=0.50 tau= 1.44 steps lag(1e-6)= 20
  rho=0.70 tau= 2.79 lag(1e-6)= 39
  rho=0.90 tau= 9.38 lag(1e-6)= 130
  rho=0.99 tau= 88.88 lag(1e-6)=1228
```

The code that produced this is a dozen lines of the notebook: build the layer, scale `weight_hh_l0`, run it, and ask autograd for the Jacobian from each intermediate state to the last.

Reading the table

- At $\rho(W) = 0.5$ the memory horizon is *one and a half steps*. A network initialised this way has forgotten its input after two steps, and no amount of training on long sequences will change that until the weights themselves grow.
- The bound uses $\sigma_{\max}(W)$, not $\rho(W)$, and the two differ: $\rho = 0.5$ has $\sigma_{\max} = 0.93$, $\rho = 0.9$ has $\sigma_{\max} = 1.7$. Between $\rho < 1 < \sigma_{\max}$ there is a regime of *transient growth with asymptotic decay* – the red curve of (a). That is where carefully initialised recurrent networks actually operate.
- None of this depends on the library. Autograd computes the product of Eq. (11.38) exactly; the vanishing gradient is *a property of the composition*, not of an implementation.

Pause and think 5: two bits per step

Question

(a) Show that the logistic function has $\max|\sigma'| = 1/4$ and tanh has $\max|\tanh'| = 1$. (b) With tanh, what is the horizon τ of Eq. (11.40) for $\sigma_{\max}(W) = 0.9$, 0.99 and 0.999, and how large must $\sigma_{\max}$ be for a horizon of 100 steps? (c) What does the theorem then say about the other side?

Pause and think 5: two bits per step

Question

(a) Show that the logistic function has $\max|\sigma'| = 1/4$ and tanh has $\max|\tanh'| = 1$. (b) With tanh, what is the horizon τ of Eq. (11.40) for $\sigma_{\max}(W) = 0.9$, 0.99 and 0.999, and how large must $\sigma_{\max}$ be for a horizon of 100 steps? (c) What does the theorem then say about the other side?

Answer

(a) $\sigma' = \sigma(1-\sigma)$ is largest at $\sigma = 1/2$, where it is $1/4$; $\tanh' = 1 - \tanh^2$ is largest at 0, where it is 1. (b) $\tau = -1/\log \sigma_{\max}$: 9.5, 99.5 and 999.5 steps. A horizon of 100 needs $\sigma_{\max} \geq e^{-1/100} \approx 0.990$. (c) The theorem's growth condition is $\mu\sigma_{\max} > 1$, and 0.99 sits one per cent below it. To remember a hundred steps the simple RNN must be tuned to within a per cent of the exploding regime – and training moves W. That is the design problem the LSTM of Part 6 solves.

Gradient clipping, Eq. (11.46)

Vanishing and exploding gradients are not symmetric problems. Explosion is the easier of the two, because it is *visible*: the gradient norm spikes and the parameters jump. The standard fix rescales any gradient whose norm exceeds a threshold, keeping the direction:

$$\mathrm{g} \longleftarrow \begin{cases} \mathrm{g}, & \|\mathrm{g}\| \le \theta, \\ \dfrac{\theta}{\|\mathrm{g}\|}\,\mathrm{g}, & \|\mathrm{g}\| > \theta. \end{cases}$$

Proposition 11.3: clipping is a trust region

(i) $\tilde{\mathrm{g}} = \lambda \mathrm{g}$ with $\lambda \in (0, 1]$, so $-\tilde{\mathrm{g}}$ is still a descent direction. (ii) $\tilde{\mathrm{g}}$ is the projection of g onto the ball of radius θ, Eq. (11.47), so a step $\gamma\tilde{\mathrm{g}}$ never moves the parameters further than $\gamma\theta$ – the guarantee of a trust-region method, for free.

Clipping: what it does not do, and where it lives

What it does not do

Nothing whatever for *vanishing* gradients: rescaling 10^{-18} upwards would amplify rounding noise, not signal. Vanishing gradients need a change of architecture (Part 6).

In the libraries

`Adam(lr, clipnorm=1.0)` in Keras; `nn.utils.clip_grad_norm_(model.parameters(), 1.0)` between `backward()` and `step()` in PyTorch. Both act on the *whole* gradient vector, all arrays jointly.

Initialisation

$\sigma_{\max}(W) \approx 1$ puts the network at the edge between the two regimes – the best this architecture can do. Orthogonal or identity initialisations of W are attempts to sit exactly there.

Clipping and initialization solve different problems

Clipping, Eq. (11.46)

Rescales the concatenated gradient to norm θ when it exceeds θ: the direction is preserved, a plain SGD step is bounded by $\gamma\theta$, and a rare explosion cannot destroy the weights. It does **not** restore a contribution that has vanished: a factor 10^{-18} is still 10^{-18} after clipping.

Initialization

An orthogonal W preserves norms in a *linear* recurrence (all singular values one). With tanh the factors $D^{(t)}$ still contract, and unconstrained updates destroy the orthogonality within a few steps. It helps at the start; it is not a guarantee. Check the input scale and the saturation alongside W (previous frames).

The honest summary

Clipping is for explosion. For vanishing there is no fix inside the simple RNN – only the architectural change of the next part, or the attention mechanism of Chapter 13.

Four effective ways to train a recurrent network

1. **Long short-term memory.** Make the RNN out of modules designed to remember values for a long time – Part 6.
2. **Hessian-free optimisation.** An optimiser that can detect directions with a tiny gradient but an even smaller curvature (Newton's method of week 37 made affordable).
3. **Echo-state networks.** Initialise the recurrent connections very carefully, so that the hidden state is a reservoir of weakly coupled oscillators – and learn only the read-out V (linear regression, week 35!).
4. **Good initialisation with momentum.** Initialise as in an echo-state network, then learn everything, with momentum and clipping.

In practice: a gated cell, $\sigma_{\max}(\mathrm{W}) \approx 1$ at the start, Adam, and clipping. Before we gate anything, let us see what the plain layer can do when the sequences are short enough.

The oscillator experiment, 1: the data and the Keras model (Section 11.6.1)

```
def make_windows(x, L=40, stride=5): # x: one trajectory of 800 points
    idx = range(0, len(x) - L - 1, stride)
    X = np.stack([x[i:i + L] for i in idx])[..., None] # (152, 40, 1): inputs x_i..x_{i+39}
    Y = np.stack([x[i + 1:i + L + 1] for i in idx])[..., None] # (152, 40, 1): targets, one step ahead
    return X, Y

model = keras.Sequential([keras.Input(shape=(None, 1)),
                          layers.SimpleRNN(24, return_sequences=True),
                          layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(5e-3, clipnorm=1.0), loss="mse")
model.fit(X_train, Y_train, epochs=300, batch_size=4, verbose=0)
```

- Data: the *single* RK4 trajectory of panel (a), $\eta = 0.2$, $(x_0, v_0) = (1, 0)$, 800 points – the network sees only $x(t)$.
- Task: one-step-ahead forecasting at every step, along 152 overlapping windows of length 40 (stride 5).
- Network: $n_h = 24$, 649 parameters; Adam at $5 \cdot 10^{-3}$, clipping at 1, 300 epochs.

The oscillator experiment, 2: the same in PyTorch

```
model = Forecaster() # nn.RNN(1, 24) + nn.Linear(24, 1) from Part 3
opt, lossfn = torch.optim.Adam(model.parameters(), lr=5e-3), nn.MSELoss()
gnorms = []
for epoch in range(300):
    for idx in torch.randperm(len(X_train)).split(4):
        opt.zero_grad()
        loss = lossfn(model(X_train[idx]), Y_train[idx])
        loss.backward()
        gnorms.append(float(nn.utils.clip_grad_norm_(model.parameters(), 1.0)))
        opt.step()
```

The tests

(b) the trajectory from $(0, 1)$ – a different solution of the same equation, out of phase with everything the network saw; (c) an oscillator driven by $F(t) = 0.3\cos(0.7t)$ from $(0.5, -0.3)$, which the network never saw in any form. The split is by whole trajectories: every test trajectory has a new initial condition.

Two rules for sequential data

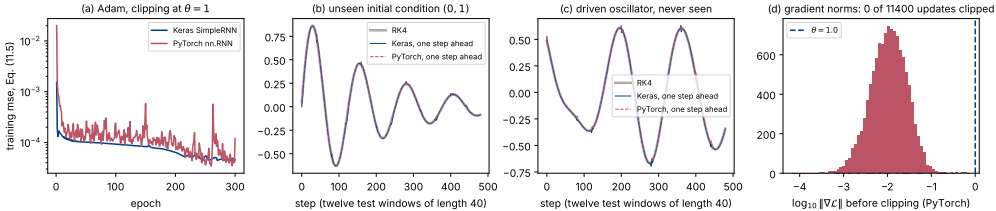
No shuffling across time

Training and test *windows* may be shuffled, the points inside a window may not – shuffling in time decorrelates the series and leaks the future into the past. `randperm(...).split(4)` and Keras's `shuffle=True` shuffle windows, never steps.

The cost

Mean squared error of predicted against actual trajectory, Eq. (11.5): a forecasting problem is a regression problem, and we are free to choose among the regression losses of week 35.

The result (week45rnn_oscillator.pdf)




```
set Keras mse PyTorch mse Var(x) relative (Keras / PyTorch)
train 3.692e-05 1.056e-04 0.0655 0.056% / 0.161%
unseen IC 3.811e-05 1.023e-04 0.0618 0.062% / 0.166%
driven 4.970e-05 1.269e-04 0.1488 0.033% / 0.085%
PyTorch gradient norms before clipping: median 0.011, max 0.77, 0 of 11400 updates clipped
```

What the result says

- The second row is the interesting one: trained on a trajectory starting from rest at unit displacement, tested on one starting from the origin at unit velocity, the error is *the same as the training error* – six parts in ten thousand of the variance in Keras, sixteen in PyTorch.
- The driven oscillator, never seen, is predicted just as well.
- The network has not memorised a trajectory; it has learned a *one-step update rule*, as Eq. (11.12) said it would – the same payoff parameter sharing gave in week 44. Whether that rule is the oscillator's, or merely "next value $\approx$ this value", the one-step test cannot say: the free rollout three frames on can.
- The two libraries differ by a factor of three in the final error: different random initialisations and different batch orderings. Both are far below anything that matters.
- (d): with $\sigma_{\max}(W) \approx 1$ at the start and sequences of 40 steps, clipping never fired in 11 400 updates. It is insurance.

Why a good one-step fit can fail in a rollout (Section 11.6.2, Eq. (11.48))

At training time the network receives *measured* states; in a *free rollout* it feeds back its own predictions. With $e_t = \hat{s}_t - s_t$ and the local model discrepancy $r_t = \hat{f}(s_t, F_t) - f(s_t, F_t)$, to first order

$$e_{t+1} \simeq J_{\hat{f}}(s_t, F_t)\, e_t + r_t.$$

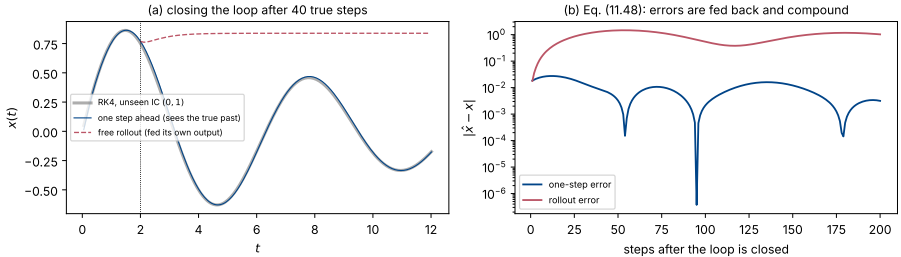
Two sources, one curve

The rollout *injects* new error (r_t) at every step and *transports* the old one ($J_{\hat{f}} e_t$) – the same Jacobian that governed the gradients now governs the forecast. A small one-step test loss says nothing about phase, amplitude or timing after many periods.

What to do

Validate on held-out trajectories; plot the error *against the forecast horizon*; compare one-step evaluation with a free rollout under the same forcing; and compare with a trusted reference (RK4), not with the Euler data of a different integrator. The next frame does this for the oscillator network we just trained.

Measured: closing the loop (week45rnn_rollout.pdf)




```
_, h = model(x_true[:40]) # warm-up on
     truth
x = x_true[39]
for _ in range(200): # close the loop
    x, h = model(x, h) # own output fed
         back
```

The same `nn.RNN` forecaster, on the unseen initial condition. Fed the true past it tracks RK4 to 10^{-2} (median one-step error 8×10^{-3}). Fed its own output it **freezes at a constant within twenty steps**: at $\Delta t = 0.05$ a map close to the identity already has a tiny one-step loss, and nothing in the training ever checked whether the network carried the *velocity* in h. The "learned rule" of the previous frames is a one-step rule; the free rollout is the test it had not been given. The remedies are in the training, not in the cell: predict increments, train on multi-step rollouts (truncated BPTT through the network's own outputs), or give the network the state it is missing.

A diagnostic for each failure mode

observation	measurement or controlled comparison
only short-delay cases work	vary the delay on the copy task; track the gradient of the final loss at each earlier state (`torch.autograd`, as in `week45rnn_gradients.pdf`)
loss spikes intermittently	log the global gradient norm *before* clipping, the clipping fraction and the update norms (panel (d) of `week45rnn_oscillator.pdf`)
many hidden units near ± 1	inspect the distribution of $1 - h_i^{(t)2}$; rescale the inputs
results depend on chunk boundaries	vary the chunk length with the task and the loss normalization fixed
one-step errors small but rollouts drift	compare phase and amplitude over increasing horizons on held-out trajectories, against a trusted integrator

Every row is a measurement, not a guess: this is how to debug a recurrent network in a project.

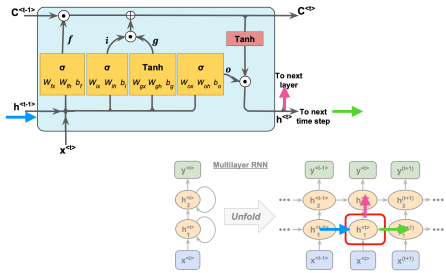
The idea: a memory cell with gates (Section 11.7)

Theorem 11.1 says the trouble is a product of Jacobians, each carrying a factor W and a factor σ'_h. Any architecture that keeps the gradient alive over long lags must *break that product*.

Hochreiter and Schmidhuber (1997) did so with a second state variable whose update is *additive* – a memory cell:

- a linear unit with a self-connection of weight 1 maintains its value indefinitely;
- information is written when a *write* gate opens, kept while a *keep* gate is open, read when a *read* gate opens.

The gates are logistic functions, hence differentiable, so the whole circuit trains by backpropagation.



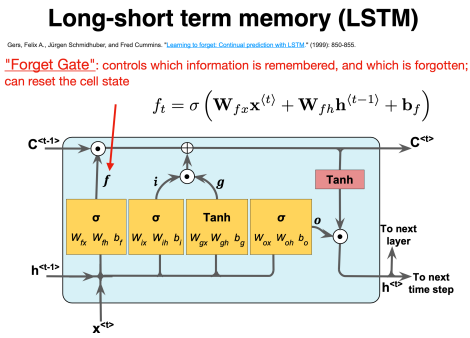
The LSTM cell of Raschka et al.: the horizontal line at the top carries $c^{(t-1)}$ to $c^{(t)}$; the four yellow boxes are the gates.

The forget gate, Eq. (11.49)

$$\mathrm{f}^{(t)} = \sigma\left(\mathrm{W}_{fx}\mathrm{x}^{(t)} + \mathrm{W}_{fh}\mathrm{h}^{(t-1)} + \mathrm{b}_f\right).$$

The logistic σ is near 0 for very negative arguments and near 1 for very positive ones: $\mathrm{f}^{(t)}$ is a soft, learned, per-component switch on the long-term memory.

The name is slightly off – $f = 1$ means *keep*.



The input gate, Eq. (11.50)

Two functions rather than one – a logistic deciding *how much* and a tanh deciding *what*:

$$\mathsf{i}^{(t)} = \sigma\Big(\mathsf{W}_{ix}\mathsf{x}^{(t)} + \mathsf{W}_{ih}\mathsf{h}^{(t-1)} + \mathsf{b}_i\Big),$$
$$\mathsf{g}^{(t)} = \tanh\Big(\mathsf{W}_{gx}\mathsf{x}^{(t)} + \mathsf{W}_{gh}\mathsf{h}^{(t-1)} + \mathsf{b}_g\Big).$$

$\mathsf{g}^{(t)}$ is a candidate value in $(-1, 1)$ – exactly the simple RNN update – and $\mathsf{i}^{(t)}$ decides how much of it is written.

The cell update, Eq. (11.51): the whole idea in one line

With $\odot$ the elementwise product,

$$\boxed{c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot g^{(t)}.}$$

Read it slowly

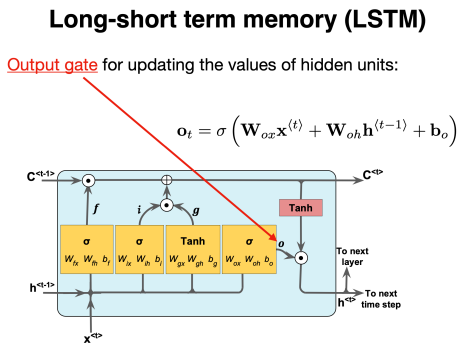
Multiplied once (by the forget gate), added to once (by the input gate). No weight matrix, no squashing function on the path from $c^{(t-1)}$ to $c^{(t)}$.

The output gate, Eq. (11.52), and the cost of all this

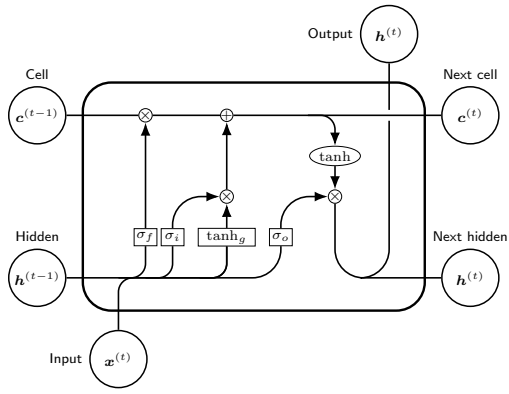
$$\mathsf{o}^{(t)} = \sigma\left(\mathsf{W}_{ox}\mathsf{x}^{(t)} + \mathsf{W}_{oh}\mathsf{h}^{(t-1)} + \mathsf{b}_o\right),$$
$$\mathsf{h}^{(t)} = \mathsf{o}^{(t)} \odot \tanh\left(\mathsf{c}^{(t)}\right).$$

$\mathsf{h}^{(t)}$ is both the output of the cell (what the read-out sees) and the input to the next time step.

Four gates, each with its own $\mathsf{W}_{\cdot x}$, $\mathsf{W}_{\cdot h}$ and bias: **four times the parameters** of a simple RNN with the same n_h, $4n_h(n_x + n_h + 1)$ – the 2496 against 624 of Part 3.



The circuit, and the constant error carousel (Figure 11.5)



Follow the horizontal line entering as $c^{(t-1)}$ and leaving as $c^{(t)}$: it meets one multiplication and one addition and nothing else. Every other path hangs *off* that line. The simple RNN would have the state line running through a matrix multiplication and a tanh at every step. The LSTM did not remove those operations; it *moved them off the memory path*.

Why it works, Eq. (11.53)

Differentiate the cell update with respect to the previous cell state, treating the gates as slowly varying:

$$\frac{\partial \mathrm{c}^{(t)}}{\partial \mathrm{c}^{(t-1)}} \approx \operatorname{diag}\left(\mathrm{f}^{(t)}\right), \qquad \frac{\partial \mathrm{c}^{(T)}}{\partial \mathrm{c}^{(t)}} \approx \prod_{k=t+1}^{T} \operatorname{diag}\left(\mathrm{f}^{(k)}\right).$$

Compare Eq. (11.38): no W, no σ_h' – the product is over the forget gates alone. If the network learns to hold $f \approx 1$ on some component, the gradient along it travels essentially undamped for as long as the gate stays open. That is the *constant error carousel*.

It is worth checking, because the argument is often stated as though the LSTM simply solves the problem – and it does not.

Measured on an untrained `nn.LSTM` (panel (b) of `week45rnn_gradients.pdf`)

$n_h = 20$, $T = 60$; the forget-gate bias set through `lstm.bias_ih_l0[n_h:2n_h] = b_f`. Two numbers per bias: the product of forget gates read from the layer, Eq. (11.53), and the exact Jacobian $\partial c^{(T)}/\partial c^{(1)}$ from autograd (Proposition 11.4):

```
forget bias mean f gate product (11.53) exact autograd (Prop. 11.4)
      0.0 0.5000 7.759e-18 3.493e-07 (simple RNN, rho=0.5: 2.9e-18)
      1.0 0.7311 4.202e-08 1.269e+00
      2.0 0.8803 2.435e-03 6.026e+03
```

- With $b_f = 0$ the gates sit at $\sigma(0) = 1/2$, the product is $2^{-59} \approx 10^{-18}$: along the cell path alone the LSTM is *no better than the simple RNN it was invented to replace*.
- Biasing the gates open buys ten orders of magnitude at $b_f = 1$ and fifteen at $b_f = 2$.
- The exact Jacobian is larger still: the gates conduct too. At $b_f = 1$ it is of order *one* – a gradient that survives sixty steps intact – and at $b_f = 2$ it is 6000: nothing here prevents explosion, so clipping stays.

Pause and think 6: the forget-gate bias

Question

PyTorch's `nn.LSTM` draws all biases uniformly from $(-1/\sqrt{n_h}, 1/\sqrt{n_h})$; Keras's `LSTM` has `unit_forget_bias=True` by default. Where does the mean forget gate start in each, what does the table predict, and what would you do about it in PyTorch?

Pause and think 6: the forget-gate bias

Question

PyTorch's `nn.LSTM` draws all biases uniformly from $(-1/\sqrt{n_h}, 1/\sqrt{n_h})$; Keras's `LSTM` has `unit_forget_bias=True` by default. Where does the mean forget gate start in each, what does the table predict, and what would you do about it in PyTorch?

Answer

PyTorch: the bias is essentially zero, so $f \approx \sigma(0) = 1/2$ and the cell path transmits $2^{-59} \approx 10^{-18}$ – the first line of the table. Keras: $b_f = 1$, $f \approx 0.73$ – the second line, ten orders of magnitude better along the cell path and a Jacobian of order one in full. In PyTorch, one line after constructing the layer: `with torch.no_grad(): lstm.bias_ih_l0[n_h:2*n_h] = 1.0` (gate order i, f, g, o). Jozefowicz et al. (2015) found this among the most reliable single adjustments – and on the spoken digits it will buy almost two percentage points. The architecture supplies a *path*; the initialisation decides whether training starts anywhere near it.

LSTM and GRU in the two libraries

```
# Keras: one word changes
model = keras.Sequential([keras.Input(shape=(T, n_x)), layers.LSTM(64), layers.Dense(10)])
# ... layers.GRU(64) ...
# PyTorch: same calls, one more output
lstm = nn.LSTM(n_x, 64, batch_first=True)
H, (h_T, c_T) = lstm(x) # H: (N, T, 64) = every h_t; h_T, c_T: (1, N, 64)
with torch.no_grad():
    lstm.bias_ih_l0[64:128] = 1.0 # forget gate open at the start, as Keras does
gru = nn.GRU(n_x, 64, batch_first=True)
H, h_T = gru(x) # no cell state
```

- Inside, both libraries compute the four gates with *one* matrix multiplication on $(\mathrm{x}^{(t)}, \mathrm{h}^{(t-1)})$ and split the result: `weight_ih_l0` has shape $(4n_h, n_x)$, in the order $(\mathrm{i}, \mathrm{f}, \mathrm{g}, \mathrm{o})$.
- `return_state=True` in Keras gives $(\mathrm{h}^{(T)}, \mathrm{c}^{(T)})$ too; pass them as `initial_state` to continue a sequence.
- Everything else – the read-out, the loss, clipping, the training loop – is unchanged from the simple RNN.

The gated recurrent unit, Eq. (11.56)

Cho et al. (2014) apply the same idea with fewer parts: no separate cell state, two gates instead of three,

$$
\begin{aligned}
\mathsf{z}^{(t)} &= \sigma\Big(\mathsf{W}_{zx}\mathsf{x}^{(t)} + \mathsf{W}_{zh}\mathsf{h}^{(t-1)} + \mathsf{b}_z\Big), \\
\mathsf{r}^{(t)} &= \sigma\Big(\mathsf{W}_{rx}\mathsf{x}^{(t)} + \mathsf{W}_{rh}\mathsf{h}^{(t-1)} + \mathsf{b}_r\Big), \\
\tilde{\mathsf{h}}^{(t)} &= \tanh\Big(\mathsf{W}_{hx}\mathsf{x}^{(t)} + \mathsf{W}_{hh}\Big(\mathsf{r}^{(t)} \odot \mathsf{h}^{(t-1)}\Big) + \mathsf{b}_h\Big), \\
\mathsf{h}^{(t)} &= \Big(1 - \mathsf{z}^{(t)}\Big) \odot \mathsf{h}^{(t-1)} + \mathsf{z}^{(t)} \odot \tilde{\mathsf{h}}^{(t)}.
\end{aligned}
$$

The *update* gate z is forget and input gate at once – what is not written is retained, by construction; the *reset* gate r controls how much of the past enters the candidate. The last line has the additive form of Eq. (11.51), so the carousel argument applies with $1 - \mathsf{z}$ in place of f.

LSTM or GRU, and what came after

Cost and performance

Three sets of matrices against the LSTM's four: three quarters of the parameters and the computation (1944 against 2496 in Part 3). Which is better is task-dependent and not settled; the two are usually close (Chung et al. 2014, Jozefowicz et al. 2015).

Still moving

Beck et al. (2024, xLSTM): exponential gating and modified memory structures make an extended LSTM competitive with transformers and state-space models. The transformer itself, which replaces recurrence by attention, is Chapter 13.

Forecasting a sine wave, 1: Keras (Section 11.8.1)

```
time_steps = np.linspace(0, 100, 500); data = np.sin(time_steps); seq_length = 20
X = np.array([data[i:i+seq_length] for i in range(len(data)-seq_length)]).reshape(-1, seq_length, 1)
y = data[seq_length:].reshape(-1, 1) # the next value after each window
split = int(0.8 * len(X)) # split in TIME, not at random

model = keras.Sequential([keras.Input(shape=(seq_length, 1)),
                          layers.SimpleRNN(16), # last state only: (N, 16)
                          layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(1e-2, clipnorm=1.0), loss="mse")
model.fit(X[:split], y[:split], epochs=200, batch_size=32, verbose=0,
          validation_data=(X[split:], y[split:]))
```

Sequence-to-label: 20 values in, *one* value out, so `return_sequences` stays at its default `False` and `Dense(1)` acts on the last state. The last 20% of the wave is the test set.

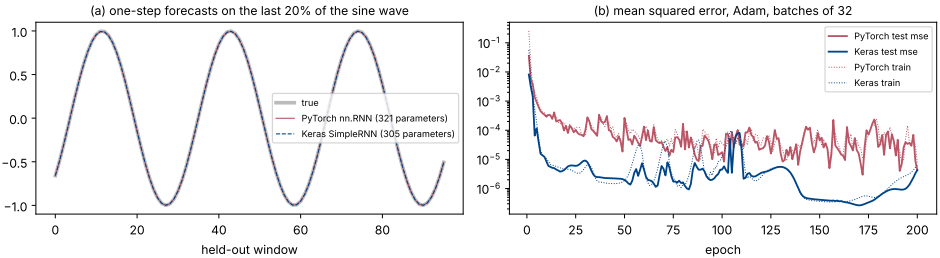
Forecasting a sine wave, 2: PyTorch

```
class SineRNN(nn.Module):
    def __init__(self, hidden=16):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)
    def forward(self, x):
        out, h_T = self.rnn(x) # out: (N, 20, 16), every h_t
        return self.fc(out[:, -1, :]) # read out the last state only

model = SineRNN(); opt = torch.optim.Adam(model.parameters(), lr=1e-2); lossfn = nn.MSELoss()
for epoch in range(200):
    for idx in torch.randperm(split).split(32):
        opt.zero_grad()
        loss = lossfn(model(X[idx]), y[idx])
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
```

`out[:, -1, :]` is Keras's `return_sequences=False`. The loop is the one of Part 3, unchanged.

Both reach the same place (`week45rnn_sine.pdf`)




```
PyTorch epoch 200: train mse 0.000008 test mse 0.000004 (321 parameters)
Keras epoch 200: train mse 0.000006 test mse 0.000004 (305 parameters)
rms error on the held-out windows: 0.0021 (both), amplitude 1
```

A test mean squared error of a few 10^{-6} is a root-mean-square error of a few 10^{-3} on a signal of unit amplitude: both predictions lie on the true curve. The parameter counts differ by sixteen: the second bias vector.

MNIST read row by row (Section 11.8.2)

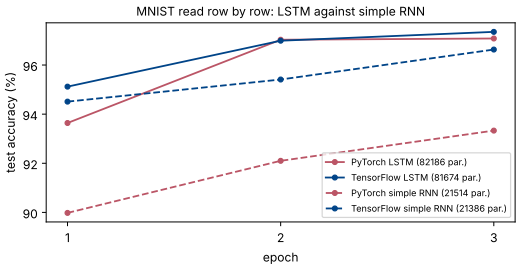
Treat each 28×28 image as a sequence of 28 rows, each of 28 features, and classify it from the last state – a sequence-to-label task. Not what one does in practice (week 44 explains why a CNN is the right tool), but a clean demonstration.

```
# Keras
m = keras.Sequential([keras.Input(shape=(28, 28)), layers.LSTM(128), layers.Dense(10)])
m.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
          loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=["accuracy"])
m.fit(x_train, y_train, epochs=3, batch_size=64) # x_train: (60000, 28, 28) in [0, 1]

# PyTorch
class RowNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.LSTM(28, 128, batch_first=True) # 28 steps of 28 features
        self.fc = nn.Linear(128, 10)
    def forward(self, x):
        return self.fc(self.rnn(x)[0][:, -1, :]) # last state -> class scores
criterion = nn.CrossEntropyLoss() # softmax + cross entropy, week 41
```

The parameter count $4n_h(n_x + n_h + 1) = 80\,384$ printed by `model.summary()` is the four-gates check of Part 3.

Row by row: the two libraries, two cells (`week45rnn_mnist_rows.pdf`)

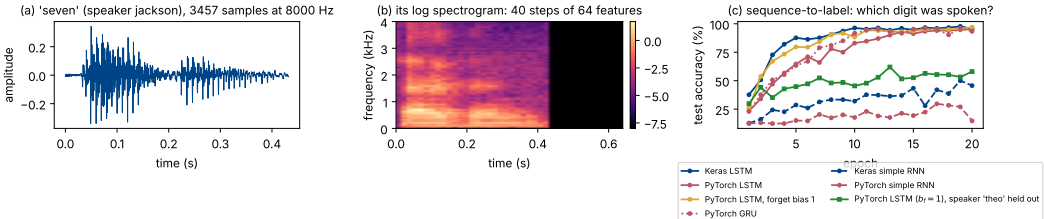



```
cell framework parameters test acc.
LSTM PyTorch 82186 0.9708
LSTM Keras 81674 0.9735
simple RNN PyTorch 21514 0.9333
simple RNN Keras 21386 0.9663
```

Three epochs, Adam 10^{-3}, batches of 64, $n_h = 128$, the full 60 000 images; accuracy on the 10 000 test images.

- The LSTM beats the simple RNN on a task with only 28 time steps – short enough that the simple RNN works, just less well.
- The whole exercise reaches about 0.97, where the convolutional network of week 44 reached 0.99: reading an image as a sequence throws away the vertical structure that a CNN was built to exploit.

Sound: spoken digits (`week45rnn_digits.pdf`, panels (a) and (b))



The Free Spoken Digit Dataset: 3000 recordings of the digits 0–9 by six speakers, 8 kHz, 0.15–2 s each (median 0.42 s). A real sequence-to-label problem with sequences of different lengths, and the data are *sound*: the notebook plays them.

Sound: from a waveform to a sequence of feature vectors

A waveform of 3457 samples is too long a sequence for a simple recurrent layer (Corollary 11.2), and the information in speech sits in how the *spectrum* changes with time. So we do what every speech system does:

```
def log_spectrogram(x, n_fft=256, hop=128, n_frames=40, n_bins=64):
    x = np.pad(x, (0, max(0, n_fft + hop*(n_frames-1) - len(x))))[:n_fft + hop*(n_frames-1)]
    frames = np.stack([x[i*hop:i*hop + n_fft] for i in range(n_frames)]) # 32 ms every 16 ms
    spec = np.abs(np.fft.rfft(frames * np.hanning(n_fft), axis=1)) ** 2 # (40, 129) power
    spec = spec[:, :2*n_bins].reshape(n_frames, n_bins, 2).mean(axis=2) # (40, 64) bands
    return np.log10(spec + 1e-8).astype("float32")
```

- Frames of 32 ms every 16 ms, a Hann window, the power spectrum (Chapter 10, week 44: the Fourier transform of a windowed signal).
- 129 frequency bins averaged in pairs to 64 bands; logarithm because loudness is logarithmic; padded with silence or cut to 40 frames (0.66 s).
- The result is a 3-tensor $(3000, 40, 64)$: *the data layout of Part 3*, $T = 40$ steps of $n_x = 64$ features – panel (b).

Sound: the models, in both libraries

```
# Keras: 40 steps of 64 features in, 10 class scores out
m = keras.Sequential([keras.Input(shape=(40, 64)), layers.LSTM(64), layers.Dense(10)])
m.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
          loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=["accuracy"])
m.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))

# PyTorch
core, head = nn.LSTM(64, 64, batch_first=True), nn.Linear(64, 10)
with torch.no_grad():
    core.bias_ih_l0[64:128] = 1.0 # the forget-gate bias of Pause 6
for epoch in range(20):
    for idx in torch.randperm(len(X_train)).split(32):
        opt.zero_grad()
        loss = nn.CrossEntropyLoss()(head(core(X_train[idx])[0][:, -1, :]), y_train[idx])
        loss.backward(); nn.utils.clip_grad_norm_(params, 1.0); opt.step()
```

The dataset's standard split: recordings 0–4 of each digit and speaker are the test set (300), the rest train (2700). One global standardisation of the features. Everything else is the MNIST code with $28 \to 40$ steps and $28 \to 64$ features.

Sound: the results (panel (c))

```
cell framework parameters test accuracy split
LSTM Keras 33674 0.9567 standard
LSTM PyTorch 33930 0.9533 standard
LSTM, b_f = 1 PyTorch 33930 0.9700 standard
GRU PyTorch 25610 0.9333 standard
simple RNN Keras 8906 0.4567 standard
simple RNN PyTorch 8970 0.1467 standard
LSTM, b_f = 1 PyTorch 33930 0.5800 speaker 'theo' held out
```

Adam 10^{-3}, batches of 32, $n_h = 64$, 20 epochs; 10 classes, so chance is 0.10.

Sound: reading the results

- A gated cell classifies a spoken digit from its spectrogram with 95–97% accuracy after twenty epochs and half a minute of training. The GRU is close behind with three quarters of the parameters.
- The simple RNN *fails*: 15% in PyTorch, 46% in Keras, against a chance level of 10%. Forty steps is beyond its memory horizon at initialisation, and the label depends on the whole utterance. This is Corollary 11.2 on real data.
- The two libraries' LSTMs agree to within half a point; setting the forget-gate bias to one in PyTorch adds almost two points on top. Pause 6 made a prediction; this is the measurement. (The Keras numbers move by a point or so between runs – TensorFlow's threading is not seeded; the PyTorch numbers are reproducible.)
- Hold out one of the six *speakers* instead of random recordings and the accuracy falls to 58%. The standard split tests whether the network recognises digits *from these speakers*; the speaker split tests whether it recognises digits.

Pause and think 7: the held-out speaker

Question

Why does accuracy fall from 97% to 58% when one speaker is held out, and what does that say about the standard split? Which of the course's tools would you use to decide which number to report?

Pause and think 7: the held-out speaker

Question

Why does accuracy fall from 97% to 58% when one speaker is held out, and what does that say about the standard split? Which of the course's tools would you use to decide which number to report?

Answer

In the standard split every test recording comes from a speaker whose other 495 recordings are in the training set: the network can use the speaker's pitch, accent and tempo as features, and it does. Held-out recordings are therefore *not independent* of the training data in the sense that matters for generalisation to a new speaker – the i.i.d. assumption of the first frame, violated at the level of the split rather than of the samples. The honest estimate for a system meant to work on new voices is the speaker split; with six speakers that is *leave-one-speaker-out cross-validation* (week 36), six folds, and the number to report is the mean and its spread. More speakers, data augmentation (pitch and tempo shifts) and a bidirectional layer are the usual ways to raise it.

The experiment the theory demands: the adding problem (Section 11.8.5)

We have seen that the LSTM keeps a larger Jacobian, and classifies better over 28 and 40 steps. Neither is the claim the architecture was invented to support: that gating lets a network learn dependencies a simple recurrence *cannot*. Testing that needs a task where nothing but long-range memory is measured.

The adding problem

Each sequence has two channels: the first carries values drawn uniformly from $[0, 1]$, the second is zero except for exactly two markers, one in each half. The target is half the sum of the two marked values.

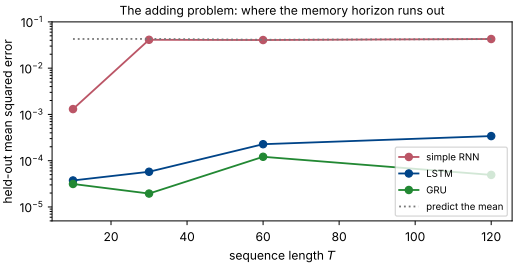
Nothing can be inferred from local statistics – the values are i.i.d. – so a model must hold one number from the first half until the second marker arrives, while ignoring everything in between. Predicting the mean gives a mean squared error of about 0.043; that is the number to beat.

The adding problem: setup

```
for T in (10, 30, 60, 120):
    for Cell in (nn.RNN, nn.LSTM, nn.GRU):
        core, head = Cell(2, 32, batch_first=True), nn.Linear(32, 1) # two channels in
        params = list(core.parameters()) + list(head.parameters())
        opt = torch.optim.Adam(params, lr=5e-3)
        for epoch in range(40):
            for idx in torch.randperm(3000).split(64):
                opt.zero_grad()
                loss = nn.MSELoss()(head(core(X[idx])[0][:, -1, :]), y[idx])
                loss.backward(); nn.utils.clip_grad_norm_(params, 1.0); opt.step()
```

- 3000 training and 1000 held-out sequences; $n_h = 32$; the same seed, optimiser and epochs for all three cells.
- The three cells are interchangeable in the code: `Cell(2, 32, batch_first=True)` and `[0][:, -1, :]` work for all of them.
- Corollary 11.2 predicts: a simple RNN initialised in the usual way has a horizon of a few steps, so a dependency at lag 30 should be invisible to it.

Where the horizon runs out (`week45rnn_adding.pdf`)



T	baseline	rnn	lstm	gru
10	0.0427	0.0013	0.0000	0.0000
30	0.0429	0.0410	0.0001	0.0000
60	0.0411	0.0405	0.0002	0.0001
120	0.0425	0.0426	0.0003	0.0000

Held-out mean squared error; the baseline column is predicting the mean.

- At $T = 10$ the simple recurrent network solves the task. At $T = 30$ it has *already failed completely* – an error equal to the baseline is a model that outputs the mean – and never recovers.
- The LSTM and the GRU solve every case, including $T = 120$, at errors two to three orders of magnitude below the simple RNN, with the same width, optimiser and epochs. The gap between $T = 10$ and $T = 30$ is where the horizon runs out.

Exercises for the lecture (pencil and a few lines of code)

1. **Linear recurrence unrolled.** Take σ_h linear and $b = 0$. Write $h^{(3)}$ in terms of $x^{(1)}, x^{(2)}, x^{(3)}, h^{(0)}$. Which power of W multiplies $Ux^{(1)}$? Then show that the Euler matrix A of Eq. (11.15) has eigenvalues of modulus $\sqrt{(1-\eta\Delta t/2)^2 + \Delta t^2 - \eta^2\Delta t^2/4}$ to leading order, and evaluate it for $\eta = 0$ and $\eta = 0.2$ at $\Delta t = 0.05$.
2. **Two steps by hand.** In the worked example set $w = 0$. Which of $\partial\mathcal{L}/\partial u$ and $\partial\mathcal{L}/\partial w$ vanishes, and why? Check with the five-line `torch` program.
3. **Weighted supervision.** In the same example give the first step a target $y_1 = 0.7$ with $\alpha_1 = \alpha_2 = 1$ and recompute δ_1. Which term of the boxed recursion appears that was zero before?
4. **The discrete adjoint.** Implement Eqs. (11.36)–(11.37) in NumPy or `torch` for the Euler oscillator and recover η from a single final position by gradient descent. Then let `torch.autograd` differentiate the same Euler loop and compare.
5. **Transient growth.** For $W = \left(\begin{smallmatrix} 0.9 & 5 \\ 0 & 0.9 \end{smallmatrix}\right)$ plot $\|W^n\|_2$ against n and find its maximum; repeat for an `nn.RNN` whose `weight_hh` is this matrix, with `torch.autograd`.
6. **Rollout.** Train the oscillator forecaster with full BPTT and with chunks of length 10 (detached states); compare the one-step error and the free-rollout error against the horizon.

Checks on the first three exercises

1. $\mathrm{h}^{(3)} = \mathrm{W}^3\mathrm{h}^{(0)} + \mathrm{W}^2\mathrm{U}\mathrm{x}^{(1)} + \mathrm{W}\mathrm{U}\mathrm{x}^{(2)} + \mathrm{U}\mathrm{x}^{(3)}$: earlier inputs pass through more applications of W. The exact eigenvalues of A are $1 - \eta\Delta t/2 \pm i\Delta t\sqrt{1-\eta^2/4}$, of modulus 1.00125 for $\eta = 0$ and 0.9962 for $\eta = 0.2$ – the numbers of the integrator frame.
2. With $w = 0$: $h_2 = \tanh(0) = 0$, $e_2 = -0.5$, $\delta_2 = -0.5$, and $\delta_1 = (1 - h_1^2)\, w\, \delta_2 = 0$. Hence $\partial\mathcal{L}/\partial u = \delta_1 x_1 + \delta_2 x_2 = 0$ while $\partial\mathcal{L}/\partial w = \delta_2 h_1 = -0.5\tanh(1) = -0.380797$: the loss cannot teach the input weight through a zero recurrent link, but it can teach the link itself to open.
3. With $\alpha_1 = 1$: $e_1 = h_1 - 0.7 = 0.061594$, and the recursion gains its first term, $\delta_1 = (v\, e_1 + w\, \delta_2)(1 - h_1^2) = (0.061594 - 0.059281) \times 0.419974 = 0.000971$ – the direct signal from the step's own output nearly cancels the one returning from the future.

Reading, code, and where this leads

Reading

Chapter 11 of the lecture notes, all sections – the material of this lecture follows it in order, including the second derivation in Section 11.3.1, the three views of BPTT in 11.4.1–11.4.7, the eigenmode discussion in 11.5.1 and the rollout in 11.6.2. Goodfellow et al. Chapter 10; Raschka et al. Chapter 15; Pascanu, Mikolov and Bengio, *On the difficulty of training recurrent neural networks*, ICML 2013 (clipping, the delayed copy); Hochreiter and Schmidhuber 1997 (LSTM); Cho et al. 2014 (GRU).

Code

Everything is in `week45rnn.ipynb`: the functions of `make_week45rnn_figures.py`, Keras and PyTorch side by side, with `mnist.npz` and `fsdd_features.npz` next to it.

In a project

A recurrent network is a reasonable choice when the data are sequences of moderate length and the past matters: time series from an experiment or a simulation, trajectories, spectra read as sequences, text. Compare it honestly with a dense network on the same windows and with a convolutional network along time; report the horizon dependence of a free rollout; use the gated cells unless the sequences are short.

What comes after

Attention and the transformer (Chapter 13) remove the sequential wall of Eq. (11.35) and the memory horizon of Corollary 11.2 at once – by letting every step look at every other step directly.

Summary

What a recurrent network is

The architecture obtained by insisting that a model respect the ordering of its data and reuse its parameters along the time axis: a learned dynamical system, and – the oscillator showed, in both directions – a learned time-stepping scheme whose linear special case *is* forward Euler. In the libraries it is one layer, `SimpleRNN` or `nn.RNN`, fed a 3-tensor `(samples, timesteps, features)`.

How it is trained, and why that is hard

Backpropagation through time in three equivalent views – the chain-rule recursion, the pre-activation error with its outer products, the path sum – checked by hand on two steps and by `torch.autograd`; the Euler case is the discrete adjoint. The gradient across a lag is a product of Jacobians: 10^{-18} over 59 steps at $\rho(\mathrm{W}) = 0.5$, a memory horizon of 1.4 steps; eigenmodes, saturation and transient growth say when to trust the bound. Clipping handles explosion, truncation trades gradient for memory, and a free rollout is the test that a one-step loss cannot replace.

What the gates change

An additive cell path whose Jacobian is a product of forget gates: order one over sixty steps when the gate starts open. A simple RNN failed on forty-step spoken digits and on the adding problem from $T = 30$; the LSTM and the GRU solved both.